

扩散策略策略优化

Allen Z. Ren¹, Justin Lidard¹, Lars L. Ankile², Anthony Simeonov²,
Pulkit Agrawal², Anirudha Majumdar¹, Benjamin Burchfiel³, Hongkai Dai³, Max Simchowitz^{2,4}

¹Princeton University ²Massachusetts Institute of Technology

³Toyota Research Institute ⁴Carnegie Mellon University

摘要

本文提出扩散策略策略优化 (Diffusion Policy Policy Optimization, DPPO)，一个算法框架，包含在连续控制与机器人学习任务中，利用强化学习 (RL) 的策略梯度 (PG) 方法微调基于扩散的策略 (如扩散策略 [20]) 的最佳实践。PG 方法在训练其他策略参数化的 RL 策略中无处不在；然而，此前推测它们对基于扩散的策略效率较低。令人惊讶的是，我们表明，在常见基准中，与其他针对基于扩散的策略的 RL 方法以及其他策略参数化的 PG 微调相比，DPPO 在微调方面实现了最强的整体性能和效率。通过实验研究，我们发现 DPPO 利用了 RL 微调与扩散参数化之间的独特协同作用，从而实现结构化且在流形上的探索、稳定的训练以及强大的策略鲁棒性。我们进一步在一系列现实场景中展示了 DPPO 的优势，包括具有像素观测的仿真机器人任务，以及在长时程、多阶段操作任务中将仿真训练的策略零样本部署到机器人硬件上。代码网站：diffusion-ppo.github.io。

1 引言

大规模预训练加上额外微调已成为语言和图像基础模型开发中的普遍流程 [12, 65, 75, 82]。尽管使用专家数据的行为克隆 [71] 正迅速成为预训练机器人策略的主导范式 [27, 28, 30, 50, 106]，但由于专家数据次优或专家数据对环境条件的覆盖有限，其性能可能次优 [64]。由于机器人策略需要与环境交互，强化学习 (RL) [90] 是进一步优化其性能、超越演示数据极限的自然选择。然而，对于参数化为扩散模型 [38] 的预训练策略，RL 微调可能较为微妙：扩散模型已成为动作策略的主要参数化方式 [20, 66, 78]，这在很大程度上归功于其高训练稳定性和表示复杂分布的能力 [39, 49, 72, 80]。

贡献 1 (DPPO)。本文提出扩散策略策略优化 (Diffusion Policy Policy Optimization, DPPO)，这是一个通用框架，并包含一组精心选择的设计决策，用于通过强化学习中流行的策略梯度方法 [85, 91] 对基于扩散的机器人学习策略进行微调。

已有文献研究了利用强化学习改进或微调基于扩散的策略 (扩散策略) [35, 74, 100]，并将策略梯度应用于微调扩散模型的非交互式应用，如文本到图像生成 [9, 21, 25]。然而，策略梯度方法被认为在训练连续控制任务的扩散策略时效率低下 [74, 103]。相反，本文表明，对于从专家演示中预训练的扩散策略，本文通过策略梯度更新进行微调的方法能够产生鲁棒、高性能的策略，并具有良好的训练行为。

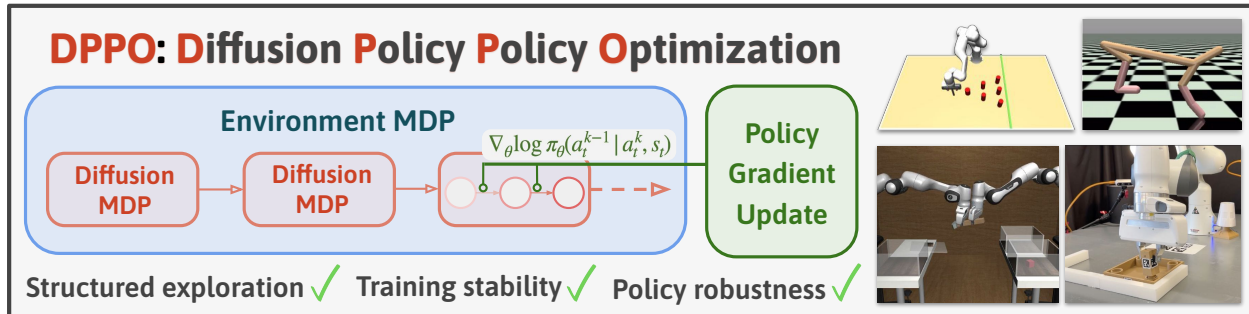


图 1: 本文提出 DPPO, 即扩散策略策略优化, 利用策略梯度对预训练的扩散策略进行微调。DPPO 将去噪过程视为马尔可夫决策过程 (“扩散 MDP”), 使任务奖励信号 (来自 “环境 MDP”) 能够轻松地通过扩散策略传播 (图 3)。策略梯度更新后利用每个去噪步骤中可处理的高斯似然。在仿真和硬件上的大量实验表明, DPPO 在策略微调期间提供了结构化探索和训练稳定性, 最终策略表现出强大的鲁棒性和泛化能力。

贡献 2 (DPPO 性能的展示)。本文表明, 对于微调预训练的扩散策略, 与一系列替代方法相比, DPPO 在不同任务上具有强大的训练稳定性, 并在具有挑战性的机器人任务中显著提升了最终性能, 这些替代方法包括基于离线 Q 学习 [35, 74, 100, 103] 和加权回归 [47, 67, 70] 的方法、其他演示增强的强化学习方法 [7, 40, 62], 以及使用策略梯度更新的常见策略参数化方法, 如高斯模型和高斯混合模型。

上述发现可能令人惊讶, 因为策略梯度方法似乎并未利用扩散采样的独特能力 (例如引导 [2, 45])。然而, 通过仔细的调查性实验, 本文发现强化学习微调与基于扩散的策略之间存在独特的协同作用。

贡献 3 (理解 DPPO 成功的机制)。我们通过大量探究性实验补充结果, 这些实验为 DPPO 强大性能背后的机制提供了洞见。与其他常见策略参数化相比, 我们提供证据表明 DPPO 进行结构化探索, 更好地利用训练数据的 “流形”, 并找到对扰动具有更强鲁棒性的策略。

通过消融实验, 我们进一步表明我们的设计决策克服了针对微调扩散策略的策略梯度方法所推测的局限性。最后, 为证明 DPPO 的广泛实用性, 我们在仿真和真实环境中验证其有效性, 并在策略输入为真实状态或像素的情况下均进行了验证。

贡献 4 (应对具有挑战性的机器人任务和设置)。我们展示 DPPO 在各种具有挑战性的机器人和控制设置中有效, 包括像素观测和稀疏奖励的长时程操作任务。我们还将通过 DPPO 在仿真中训练的策略零样本部署到真实硬件上, 与基线相比, 其仿真到现实差距显著更小。

机器人学之外的潜在影响。 DPPO 是一个通用框架, 可潜在地应用于机器人学之外序列交互设置中基于扩散模型的微调。这些包括: 将基于扩散的文本到图像生成 [9, 21] 扩展到带有人类反馈的多轮交互设置; 药物设计/发现应用 [42, 58], 在分子层面与仿真器反馈进行策略搜索 (秉承先前非基于扩散的强化学习药物发现 [73] 的精神); 以及将基于扩散的语言建模 [56, 83] 适配到交互式 (例如带有人类反馈 [65])、问题求解和规划任务。

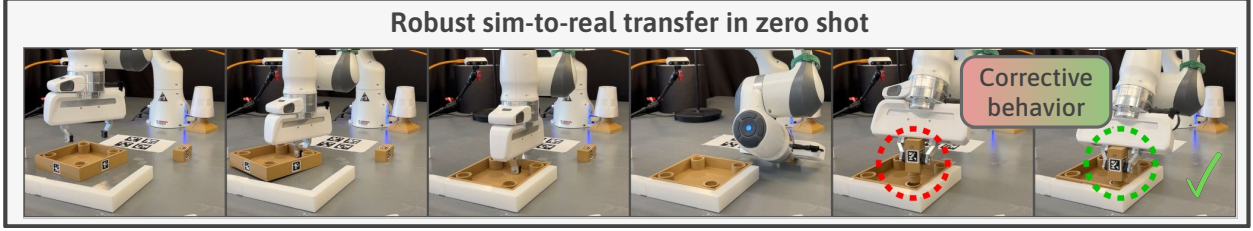


图 2: DPPO 解决来自 FURNITURE-BENCH ([36]) 的具有挑战性的长时程操作任务, 并允许在不使用任何真实数据的情况下实现鲁棒的仿真到现实迁移 (见第 5.4 节)。

1.1 方法概述

与先前工作 [9, 21] 类似, 本文方法将去噪扩散过程展开为动作似然显式的马尔可夫决策过程 (MDP); 在此基础上, 构建一个双层 MDP, 其外层对应环境 MDP, 内层对应去噪 MDP。鉴于强化学习训练的挑战, 本文还提出了一系列针对 DPPO 的设计决策, 这些决策对于在具有挑战性的机器人环境中实现 DPPO 的性能至关重要, 详见贡献 2 和贡献 4, 包括:

- 本文将近端策略优化 (PPO) [85] 应用于双层 MDP, 并展示了如何高效估计 PPO 更新所需的优势函数。
- 本文表明, 对于微调任务, 通常仅需微调去噪过程的最后几步, 或改为微调去噪扩散隐式模型 (DDIM) [87] 即可。
- 本文提出对扩散噪声调度进行修改, 以确保训练稳定性和充分探索, 同时利用扩散模型固有的随机性。

本文方法在第 4 节中正式阐述。如前所述, 本文进行了大量实验, 在第 5 节中展示了 DPPO 的成功, 并在第 6 节中给出了可能的解释。

2 相关工作

策略优化及其在机器人中的应用。策略优化方法通过动作似然的梯度来更新强化学习策略的显式表示——通常由神经网络参数化。继开创性的策略梯度 (PG) 方法 [91, 101] 之后, 出现了一系列进一步提升训练稳定性和样本效率的算法, 如 DDPG[54] 和 PPO[84]。PG 方法在训练机器人策略方面已广泛有效 [4, 17, 43, 48], 这主要归功于其高维连续动作空间中的训练稳定性, 以及在并行模拟环境中的良好扩展性。鉴于长时程任务中从零开始探索的挑战, PG 在微调由示范训练得到的基线策略方面取得了巨大成功 [68, 76, 93]。本文实验发现, 在操作任务中, 尤其是在长时程和高维动作空间的任務中, 采用在线策略 PG 的 DPPO 通常比离线策略 Q 学习方法 [7, 40, 62] 取得更强的最终性能。关于机器人策略 PG 微调及相关方法的扩展讨论见附录 A.1。

学习和改进基于扩散的策略。基于扩散的策略 [5, 20, 66, 78, 89, 99, 105] 近期在机器人技术和决策应用中取得了成功。通常, 这些策略通过监督目标从人类演示中训练, 具有较高的训练稳定性和在建模复杂及多模态轨迹分布方面的强大性能。

由于演示数据往往有限和/或次优，已提出许多方法来提高基于扩散的策略的性能。一种流行的方法是利用奖励信号或目标条件等目标来引导扩散去噪过程 [2, 15, 45, 53, 96]。更近期的工作探索了包括 Q 学习和加权回归在内的技术可能性，无论是从纯离线估计 [16, 22, 100]，还是结合在线交互 [35, 47, 74, 103]。有关这些方法的详细描述，请参见附录 A.2。

通过扩散模型的策略梯度。强化学习技术已被用于微调扩散模型，例如用于文本到图像生成的模型 [9, 24, 25, 97]。Black 等人 [9] 将去噪过程视为马尔可夫决策过程并应用 PPO 更新。我们基于这些早期发现，将去噪马尔可夫决策过程嵌入到控制任务中动力学的环境马尔可夫决策过程中，形成两层“扩散策略马尔可夫决策过程”。尽管 Psenka 等人 [74] 已经展示了如何通过两个马尔可夫决策过程中传播策略梯度来对扩散策略进行策略梯度计算，但他们推测由于去噪步骤导致的有效时间跨度增加引起的大动作方差，这种方法可能无效。我们的结果在微调设置中反驳了这一假设。

3 预备知识

马尔可夫决策过程。我们考虑一个马尔可夫决策过程 (MDP) $\mathcal{M}_{\text{ENV}} := (\mathcal{S}, \mathcal{A}, P_0, P, R)$ ，具有状态 $s \in \mathcal{S}$ 、动作 $a \in \mathcal{A}$ 、初始状态分布 P_0 、转移概率 P 和奖励 R 。在每个时间步 t ，智能体（例如机器人）观察状态 $s_t \in \mathcal{S}$ ，采取动作 $a_t \sim \pi(a_t | s_t) \in \mathcal{A}$ ，根据 $s_{t+1} \sim P(s_{t+1} | s_t, a_t)$ 转移到下一个状态 s_{t+1} ，同时获得奖励 $R(s_t, a_t)$ ²。固定马尔可夫决策过程 $\mathcal{M}_{\text{ENV}}$ ，我们令 $\mathbb{E}^\pi$ （分别地 $\mathbb{P}^\pi$ ）表示长度为 $T + 1$ 的轨迹 $(s_0, a_0, \dots, s_T, a_T)$ 上的期望（分别地概率分布），具有初始状态分布 $s_0 \sim P_0$ 和转移算子 P 。我们的目标是训练一个策略来优化累积奖励，并通过函数 $\gamma(\cdot)$ 进行折扣。

$$\mathcal{J}(\pi_\theta) = \mathbb{E}^{\pi_\theta, P_0} \left[\sum_{t \geq 0} \gamma(t) R(s_t, a_t) \right]. \quad (3.1)$$

策略优化。策略梯度方法（例如，强化算法 [101]）通过近似该目标关于策略参数的梯度来提升策略性能：

$$\nabla_\theta \mathcal{J}(\pi_\theta) = \mathbb{E}^{\pi_\theta, P_0} \left[\sum_{t \geq 0} \nabla_\theta \log \pi_\theta(a_t | s_t) r_t(s_t, a_t) \right], \quad r_t(s_t, a_t) := \sum_{\tau \geq t} \gamma(\tau) R(s_\tau, a_\tau), \quad (3.2)$$

其中 r_t 是从时间 t 开始的折扣累积未来奖励（更一般地， r_t 可以被 Q 函数估计器替代 [91])， γ 是依赖于时间步的折扣因子， $\nabla_\theta \log \pi_\theta(a_t | s_t)$ 表示 $a_t | s_t$ 似然对数的梯度。为了降低梯度估计的方差，可以学习状态值函数 $\hat{V}^{\pi_\theta}(s_t)$ 来近似 $\mathbb{E}[r_t]$ 。估计的优势函数 $\hat{A}^{\pi_\theta}(s_t, a_t) := r_t(s_t, a_t) - \hat{V}^{\pi_\theta}(s_t)$ 替代 $r_t(s_t, a_t)$ 。

¹ 更一般地，我们可以将环境视为部分可观测马尔可夫决策过程 (POMDP)，其中智能体的动作依赖于状态 s 的观测 o （例如，来自像素的动作）。我们的实现适用于此设置，但为了简洁，我们在形式化中省略了额外的观测。² 为简单起见，我们重载 $R(\cdot, \cdot)$ 以同时表示随机变量奖励及其分布。

扩散模型。去噪扩散概率模型 (DDPM) [38, 63, 86] 将连续值数据分布 $p(\cdot) = p(x^0)$ 表示为前向加噪过程 $q(x^k|x^{k-1})$ 的反向去噪过程, 该过程迭代地向数据添加高斯噪声。反向过程由神经网络 $\varepsilon_\theta(x_k, k)$ 参数化, 预测将 x_0 转换为 x_k [38] 的添加噪声 ε 。采样从随机样本 $x^K \sim \mathcal{N}(0, I)$ 开始, 并迭代生成去噪样本:

$$x^{k-1} \sim p_\theta(x^{k-1}|x^k) := \mathcal{N}(x^{k-1}; \mu_k(x^k, \varepsilon_\theta(x^k, k)), \sigma_k^2 I). \quad (3.3)$$

上述中, $\mu_k(\cdot)$ 是一个固定函数, 独立于 θ , 将 x^k 和预测的 ε_θ 映射到下一个均值, σ_k^2 是遵循从 $k = 1, \dots, K$ 开始的固定计划的方差项。我们建议读者参考 Chan[14] 的深入综述。

扩散模型作为策略。扩散策略 (Diffusion Policy, DP; 参见 Chi 等人 [20]) 是一种由去噪扩散概率模型 (DDPM) 参数化的策略 π_θ , 它以 s 作为条件参数, 并参数化 $p_\theta(a^{k-1} | a^k, s)$ as in Eq. (3.3). DP 可以通过行为克隆进行训练, 即拟合条件噪声预测 $\varepsilon_\theta(a^k, s, k)$ 来预测添加的噪声。注意, 与更标准的策略参数化 (如单峰高斯策略) 不同, DP 不维护 $p_\theta(a^0 | s)$ 的显式似然。本文采用常见做法, 训练 DP 预测动作块——未来几个时间步 (记为 T_p) 的动作序列——以提升时序一致性。策略执行预测块中的 $T_a \leq T_p$ 步后再进行下一次预测。从现在起, $a \in \mathcal{A}$ 表示整个执行的动作块。为公平比较, 除非另有说明, 我们的扩散和非扩散基线均使用相同的 T_a 。

4 DPPO: 扩散策略的策略优化

如第 1 节和第 2 节所述, 针对扩散策略的微调以提升性能已受到广泛关注 [74, 103], 主要聚焦于离线 Q 学习 [35, 100] 和/或加权回归。DPPO 采用不同方法: 对动作似然求导并应用如式 (3.2) 的策略梯度 (PG) 更新。

4.1 两层“扩散策略 MDP”

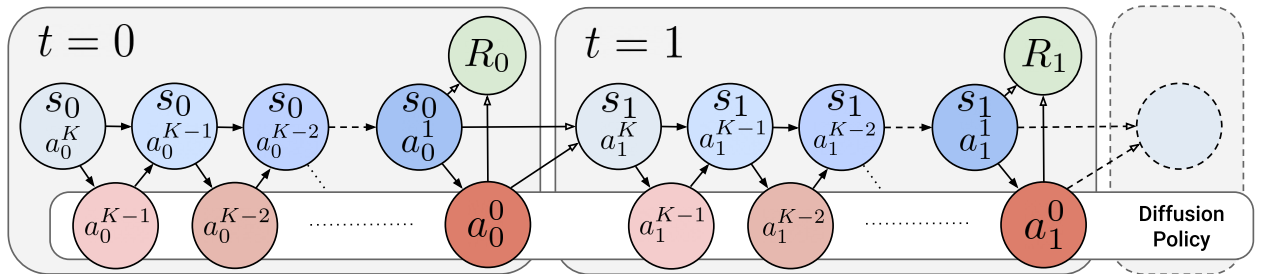


图 3: 我们将扩散策略中的去噪过程视为一个 MDP, 整个环境回合可视为一系列此类 MDP 的链。现在整条链 (“扩散策略 MDP”, $\mathcal{M}_{DP}$) 在每个 (去噪) 步骤都包含高斯似然, 因此可通过策略梯度进行优化。蓝圈表示 $\mathcal{M}_{DP}$ 中的状态, 红圈表示动作。

如 [9] 和 [74] 所述, 去噪过程可表示为一个多步 MDP, 其中每个去噪步骤的策略似然可直接获得。我们扩展此形式化方法, 将扩散 MDP 嵌入环境 MDP, 得到一个更大的“扩散策略 MDP”, 记为 $\mathcal{M}_{DP}$,

如图 3 所示。下文使用符号 δ 表示狄拉克分布， $\otimes$ 表示乘积分布。回顾第 3 节中的环境 MDP $\mathcal{M}_{\text{ENV}} := (\mathcal{S}, \mathcal{A}, P_0, P, R)$ 。扩散 MDP $\mathcal{M}_{\text{DP}}$ 使用索引 $\bar{t}(t, k) = tK + (K - k - 1)$ 对应于 (t, k) ，该索引在 t 中递增，但（为保持扩散的索引约定）随 $K - 1 \geq k \geq 0$ 按字典序递减。状态、动作和奖励为

$$\bar{s}_{\bar{t}(t, k)} = (s_t, a_t^{k+1}), \quad \bar{a}_{\bar{t}(t, k)} = a_t^k, \quad \bar{R}_{\bar{t}(t, k)}(\bar{s}_{\bar{t}(t, k)}, \bar{a}_{\bar{t}(t, k)}) = \begin{cases} 0 & k > 0 \\ R(s_t, a_t^0) & k = 0 \end{cases},$$

其中在 $\bar{t}(t, k)$ 处的柱动作是经过一步去噪后的动作 a_t^k 。奖励仅在对应于 a_t^0 被采取的时刻给出。初始状态分布为 $\bar{P}^0 = P_0 \otimes \mathcal{N}(0, \mathbf{I})$ ，对应于 $s_0 \sim P_0$ 是来自环境马尔可夫决策过程的初始分布，且 $a_0^K \sim \mathcal{N}(0, \mathbf{I})$ 相互独立。最后，转移为

$$\bar{P}(\bar{s}_{\bar{t}+1} | \bar{s}_{\bar{t}}, \bar{a}_{\bar{t}}) = \begin{cases} (s_t, a_t^k) \sim \delta_{(s_t, a_t^k)} & \bar{t} = \bar{t}(t, k), k > 0 \\ (s_{t+1}, a_{t+1}^K) \sim P(s_{t+1} | s_t, a_t^0) \otimes \mathcal{N}(0, \mathbf{I}) & \bar{t} = \bar{t}(t, k), k = 0 \end{cases}.$$

也就是说，当 $k > 0$ 时，转移将去噪后的动作 a_t^k 在步骤 $\bar{t}(t, k)$ 处移入下一状态，否则以 $k = 0$ 推进环境马尔可夫决策过程动态。纯噪声 a_t^K 在 $k = 0$ 处转移时被视为环境的一部分。根据式 (3.3)， $\mathcal{M}_{\text{DP}}$ 中的策略具有如下形式

$$\bar{\pi}_{\theta}(\bar{a}_{\bar{t}(t, k)} | \bar{s}_{\bar{t}(t, k)}) = \pi_{\theta}(a_t^k | a_t^{k+1}, s_t) = \mathcal{N}(a_t^k; \mu(a_t^{k+1}, \varepsilon_{\theta}(a_t^{k+1}, k + 1, s_t)), \sigma_{k+1}^2 \mathbf{I}). \quad (4.1)$$

幸运的是，式 (4.1) 是一个高斯似然，可以解析求值，并且适用于策略梯度更新³：

$$\nabla_{\theta} \bar{\mathcal{J}}(\bar{\pi}_{\theta}) = \mathbb{E}^{\bar{\pi}_{\theta}, \bar{P}, \bar{P}^0} \left[\sum_{\bar{t} \geq 0} \nabla_{\theta} \log \bar{\pi}_{\theta}(\bar{a}_{\bar{t}} | \bar{s}_{\bar{t}}) \bar{r}(\bar{s}_{\bar{t}}, \bar{a}_{\bar{t}}) \right], \quad \bar{r}(\bar{s}_{\bar{t}}, \bar{a}_{\bar{t}}) := \sum_{\tau \geq \bar{t}} \gamma(\tau) \bar{R}(\bar{s}_{\tau}, \bar{a}_{\tau}). \quad (4.2)$$

评估上述内容涉及通过去噪过程进行采样，这是扩散策略中采样动作的常规“前向传递”；如上所述，初始状态可以通过 $\bar{P}^0 = P_0 \otimes \mathcal{N}(0, \mathbf{I})$ 从环境中采样，其中 P_0 来自环境马尔可夫决策过程。

4.2 使用近端策略优化实例化扩散近端策略优化

我们应用近端策略优化（Proximal Policy Optimization, PPO）[1, 23, 41, 85]，这是对原始策略梯度更新的一种流行改进。完整的伪代码及实现细节见附录 B 中的算法 1。

定义 4.1（广义近端策略优化，裁剪变体）。考虑一个一般的马尔可夫决策过程。给定优势估计器 $\hat{A}(s, a)$ ，近端策略优化更新 [85] 是对以下目标的样本近似

$$\nabla_{\theta} \mathbb{E}^{(s_t, a_t) \sim \pi_{\theta_{\text{old}}}} \min \left(\hat{A}^{\pi_{\theta_{\text{old}}}}(s_t, a_t) \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}, \hat{A}^{\pi_{\theta_{\text{old}}}}(s_t, a_t) \text{clip} \left(\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}, 1 - \varepsilon, 1 + \varepsilon \right) \right),$$

其中 ε 为裁剪比率，控制策略相对于先前策略更新的最大幅度。

³[74] 提出了类似的推导，但未将去噪过程视为马尔可夫决策过程（MDP）。进一步说明见附录 A。

我们在扩散 MDP 中实例化 PPO，使用 $(s, a, t) \leftarrow (\bar{s}, \bar{a}, \bar{t})$ 。我们的优势估计器采用特定形式，以体现 MDP 的两层特性：设 $\gamma_{\text{ENV}} \in (0, 1)$ 为环境折扣因子， $\gamma_{\text{DENOISE}} \in (0, 1)$ 为去噪折扣因子。考虑环境折扣回报：

$$\bar{r}(\bar{s}_{\bar{t}}, \bar{a}_{\bar{t}}) := \sum_{t' \geq t} \gamma_{\text{ENV}}^{t'} \bar{r}(\bar{s}_{\bar{t}(t', 0)}, \bar{a}_{\bar{t}(t', 0)}), \quad \bar{t} = \bar{t}(t, k),$$

因为 $\bar{R}(\bar{t}) = 0$ 在 $k > 0$ 处。这一事实也消除了 $k > 1$ 处估计价值的需要，并允许我们使用以下去噪折扣优势估计器⁴：

$$\hat{A}^{\pi_{\theta_{\text{old}}}}(\bar{s}_{\bar{t}}, \bar{a}_{\bar{t}}) := \gamma_{\text{DENOISE}}^k \left(\bar{r}(\bar{s}_{\bar{t}}, \bar{a}_{\bar{t}}) - \hat{V}^{\pi_{\theta_{\text{old}}}}(\bar{s}_{\bar{t}(t, 0)}) \right)$$

去噪折扣的作用是降低噪声较大步骤（较大的 k ）对策略梯度的贡献（研究见附录 C.2）。最后，我们选择价值估计器仅依赖于“分量”，

$$\hat{V}^{\pi_{\theta_{\text{old}}}}(\bar{s}_{\bar{t}(t, 0)}) := \tilde{V}^{\pi_{\theta_{\text{old}}}}(s_t),$$

我们发现，与同时估计应用去噪动作 $a_t^{k=1}$ （ $\bar{s}_{\bar{t}(t, 0)}$ 的一部分）的价值相比，这能带来更高效、更稳定的训练，如附录 C.2 所示。

4.3 DPPO 的最佳实践

仅微调最后几个去噪步骤。 扩散策略通常使用多达 $K = 100$ 个去噪步骤（采用 DDPM）以更好地捕捉专家演示的复杂数据分布。使用 DPPO 时，我们可以选择仅微调去噪步骤的一个子集，例如最后 K' 步。附录 C.2 中的实验结果表明，这能加速 DPPO 训练并减少 GPU 内存占用，同时不牺牲最终性能。我们不是微调预训练模型权重 θ ，而是复制一份 θ_{FT} —— θ 被冻结并用于早期去噪步骤，而 θ_{FT} 用于最后 K' 步并通过 DPPO 更新。

微调去噪扩散隐式模型 (DDIM) 采样。 与微调所有 K 或 DDPM 的最后几步不同，也可以在微调期间应用去噪扩散隐式模型 (DDIM) [87]，这大大减少了采样步数 $K^{\text{DDIM}} \ll K$ ，例如少至 5 步，从而可能提高 DPPO 效率，因为微调的步数更少：

$$x^{k-1} \sim p_{\theta}^{\text{DDIM}}(x^{k-1}|x^k) := \mathcal{N}(x^{k-1}; \mu^{\text{DDIM}}(x^k, \varepsilon_{\theta}(x^k, k)), \eta \sigma_k^2 \mathbf{I}), \quad k = K^{\text{DDIM}}, \dots, 0. \quad (4.3)$$

尽管 DDIM 通常通过在式 (4.3) 中设置 $\eta = 0$ 用作确定性采样器，但我们可以使用 $\eta > 0$ 进行微调，以提供探索噪声并避免计算具有狄拉克分布的高斯似然。在实践中，我们设置 $\eta = 1$ 用于训练（相当于应用 DDPM[87]），然后设置 $\eta = 0$ 用于评估。我们在基于像素的实验和长时程家具装配任务中使用 DDIM 采样，这些任务非常需要效率提升。

扩散噪声调度。 我们使用 [63] 中引入的 σ_k 余弦调度，该调度最初在 $k = 0$ 处退火到 $1e-4$ 量级的小值。在 DPPO 中， σ_k 的值也转化为对训练效率至关重要的探索噪声。经验上，我们发现在采样动作时将 σ_k 裁剪到更高的最小值（记为 $\sigma_{\min}^{\text{exp}}$ ，e.g., 0.01 – 0.1）有助于探索（见附录 C.2 中的研究）。此外，在评估高斯似然 $\log^{-1} \pi_{\theta}(\bar{a}_{\bar{t}}|\bar{s}_{\bar{t}})$ 时，我们将 σ_k 裁剪为至少 0.1（记为 $\sigma_{\min}^{\text{prob}}$ ），通过避免大幅度来提高训练稳定性。

⁴ 在实践中，我们使用广义优势估计 (GAE, Schulman 等人 [84])，它在估计优势时更好地平衡了方差和偏差。这里我们呈现更简单的形式。

网络架构。我们研究多层感知机 (MLP) 和 UNet[81] 作为扩散策略中的策略头。MLP 提供了更简单的设置，我们发现它通常使用 DPPO 微调更稳定。同时，UNet 仅对 a_t^k 应用卷积，其好处是允许使用 $T_a < T_p$, e.g., $T_p = 16$ 和 $T_a = 8$ 进行微调。我们发现 DPPO 受益于使用更大的 T_p 进行预训练（更好的预测）和使用更小的 T_a 进行微调（更适合策略梯度）⁵。

5 DPPO 的性能评估 我们研究了 DPPO 在流行的强化学习和机器人基准测试环境中的性能。我们将 DPPO 与用于微调扩散策略的其他强化学习方法（第 5.1 节）、其他演示增强强化学习方法（第 5.2 节）、其他使用策略梯度更新的策略参数化方法（第 5.3 节）进行了比较，然后在包括硬件评估在内的多阶段操作任务中进行了比较（第 5.4 节），最后以消融研究作为总结（第 5.5 节）。虽然我们的评估主要集中在微调上，但我们也在附录 C 中展示了从头开始训练的结果。墙钟时间在附录 D 中报告和讨论；它们与其他基于扩散的强化学习基线大致相当（通常更快），比其他演示增强强化学习基线快得多，但可能比其他策略参数化方法慢 $2\times$ 。完整的训练超参数选择和额外的训练细节在附录 E 中给出。

环境：OpenAI Gym。我们首先考虑强化学习文献中常见的三个 OpenAI GYM 运动基准 [11]：{Hopper-v2, Walker2D-v2, HalfCheetah-v2}。所有策略都使用 D4RL[29] 中的完整中等水平数据集进行预训练，状态输入和动作块大小为 $T_a = 4$ 。我们在微调中使用原始的密集奖励设置。

环境：Franka Kitchen。我们还考虑了首次在 [32] 中引入的 FRANKA-KITCHEN 环境，该环境涉及一个 Franka 机械臂完成四个台面任务。我们使用 D4RL[29] 中的三个数据集：{Kitchen-Complete-v0, Kitchen-Partial-v0, Kitchen-Mixed-v0}，其中包含不同完整性水平的演示。考虑状态输入，DPPO 使用动作块大小 $T_a = 4$ 。我们在微调中使用原始的稀疏奖励设置。

环境：Robomimic。接下来，我们考虑来自 ROBOMIMIC 基准 [60] 的四个模拟机器人操作任务，{Lift, Can, Square, Transport}，按难度递增排序。这些任务更能代表现实世界的机器人任务，而 Square 和 Transport（图 4）被认为对强化学习训练来说非常具有挑战性。同时考虑状态和像素策略输入。基于状态和基于像素的策略分别使用 ROBOMIMIC 提供的 300 和 100 个演示进行预训练。我们主要考虑 ROBOMIMIC 中噪声较大的多人 (MH) 数据，但在一些实验中也考虑了更干净的专业人类 (PH) 数据。对于 Can、Lift 和 Square，我们考虑 $T_a = 4$ ，对于 Transport，我们考虑 $T_a = 8$ 。然后使用任务完成时等于 1 的稀疏奖励对它们进行微调。

环境：家具基准（Furniture-Bench）与真实家具装配。最后，我们展示了从家具基准 (FURNITURE-BENCH) [36] 中求解更长时域、多阶段机器人操作任务的能力。我们考虑了三个仿真家具装配任务：{单腿、台灯、圆桌}，如图 4 所示，并在附录 E.8 中详细描述。我们考虑了初始状态分布的两种随机性级别：低 (Low) 和中 (Med)，由基准定义。所有策略均使用在仿真中收集的 50 条人类演示进行预训练，并 $T_a = 8$ 。随后使用稀疏奖励（任务阶段完成指示）进行微调。我们还评估了单腿任务的零样本仿真到真实 (sim-to-real) 性能。

⁵ 在多层感知机 (MLP) 中使用全连接层时，我们经验性地发现使用 $T_a < T_p$ 进行微调会导致训练不稳定。

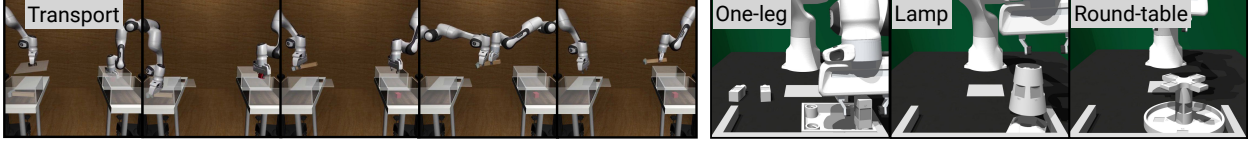


图 4: 长时域机器人操作任务, 包括 (左) 来自机器人模仿 (ROBOMIMIC) 的双臂搬运任务和 (右) 家具基准 (FURNITURE-BENCH) 任务 (完整轨迹可视化见图 A12)。

5.1 与基于扩散的强化学习算法的比较

我们将 DPPO 与一系列用于微调基于扩散策略的强化学习方法进行了比较。基线名称与其绘图颜色相对应。其中两种方法 DRWR 和 DAWR 是我们自己提出的新基线, 分别基于奖励加权回归 (RWR, Peters 和 Schaal [70]) 和优势加权回归 (AWR, Peng 等人 [67])。其余方法 DIPO [103]、IDQL [35]、DQL [100] 和 QSM [74] 均来自现有文献。其中, QSM 和 DIPO 专为从头训练而提出, 而 IDQL 和 DQL 更适合在线微调。我们在三个 OpenAI GYM 任务和四个 ROBOMIMIC 任务上使用状态输入进行评估; 所有基线的详细描述和训练细节见附录 E.3。总体而言, DPPO 表现一致, 训练稳定性极佳, 并且在各任务中展现出强大的微调性能。在 GYM 任务中 (图 5, 顶行), IDQL 和 DIPO 表现出具有竞争力的性能, 而其他方法通常表现较差且训练稳定性不足。基线方法在微调初期也往往出现性能下降, 这很可能是由于对预训练策略的 Q 值估计不准确所致。DPPO 在 ROBOMIMIC 任务中表现最为出色 (图 5, 底行), 尤其是在具有挑战性的 Transport 任务中。IDQL 以及令人意外的 DRWR 在 {Lift, Can, Square} 任务中是强基线, 但在 Transport 任务中表现不佳, 而所有其他基线表现更差。我们推测, 其他基线均采用离线策略更新并将梯度从不完美的 Q 函数传播到策略网络, 在稀疏奖励的 ROBOMIMIC 任务中, 由于连续动作空间加上较大的动作块尺寸, 会遭受更严重的训练不稳定性 (进一步研究见附录 C.2)。

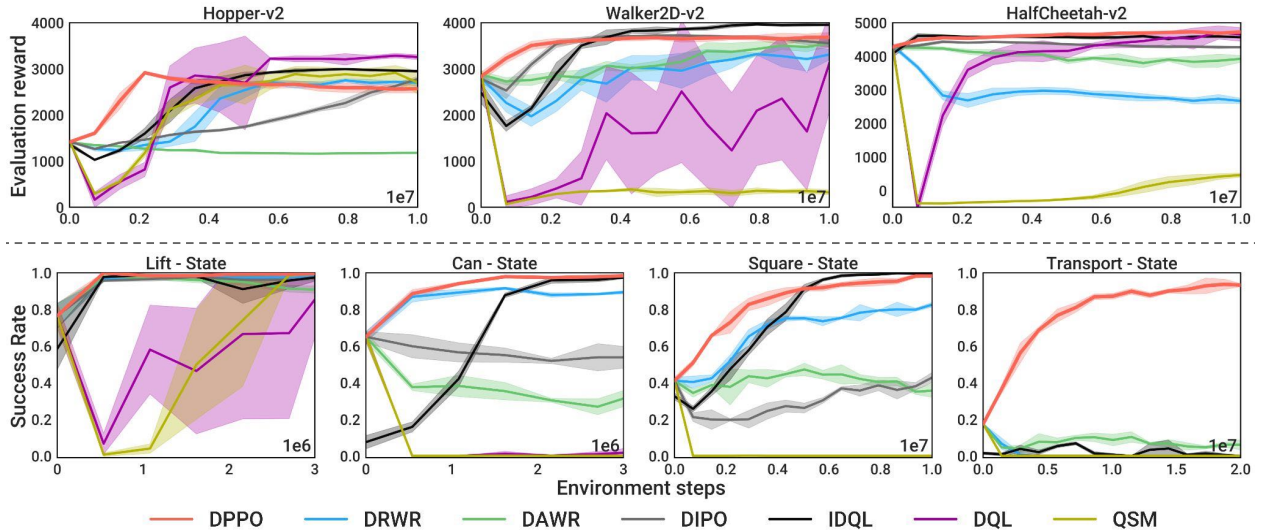


图 5: 与其他基于扩散的强化学习算法的比较。顶行: GYM 任务 [11], 在五个随机种子上取平均。底行: ROBOMIMIC 任务 [60], 在三个随机种子上取平均, 使用状态观测。DPPO 曲线略微加粗以便更清晰可见。

5.2 与其他演示增强强化学习算法的比较

我们将 DPPO 与近期提出的利用离线数据训练机器人策略（不一定是基于扩散的）的强化学习方法进行了比较，包括 RLPD [7]、**Cal-QL** [62] 和 IBRL [40]。这些方法在回放缓冲区中添加专家数据并进行离线策略更新（IBRL 和 **Cal-QL** 还分别使用行为克隆和离线强化学习目标进行预训练），在 HalfCheetah-v2 中显著提高了样本效率（图 6，左上）。在 FRANKA-KITCHEN 设置中（图 6，右上），我们发现 RLPD 和 IBRL 学习效果不佳，尤其是在来自 Kitchen-Partial-v0 和 Kitchen-Mixed-v0 的噪声较大的演示数据上。**Cal-QL** 取得了有竞争力的性能，但 DPPO 仍然取得了整体最佳性能，尤其是在 Kitchen-Complete-v0 上。我们注意到，DPPO 在微调期间不使用任何专家数据，因此可能对预训练性能敏感；我们发现 Kitchen-Partial-v0 和 Kitchen-Mixed-v0 中的不完整演示即使在扩散参数化下也难以完全建模数据的多模态性，从而阻碍了 DPPO 实现（接近）完美的微调性能。尽管如此，我们相信来自 ROBOMIMIC 的专家演示最能反映机器人技术中的预训练加微调，因为所有演示都完成了任务，尽管质量参差不齐。图 6 底行显示了 DPPO 和基线在 Can 和 Square 中使用更干净的 PH 或噪声更大的 MH 数据时的性能；DPPO 表现出更强的最终性能。RLPD 和 **Cal-QL** 完全无法学习，而 IBRL 在较低的成功率水平上饱和。DPPO 在墙钟时间上也比基线快得多，因为它利用了高度并行化环境的采样⁶；因此，我们将 ROBOMIMIC 中基线的样本数上限设为 $1e6$ ，也是因为它们的性能已经饱和。

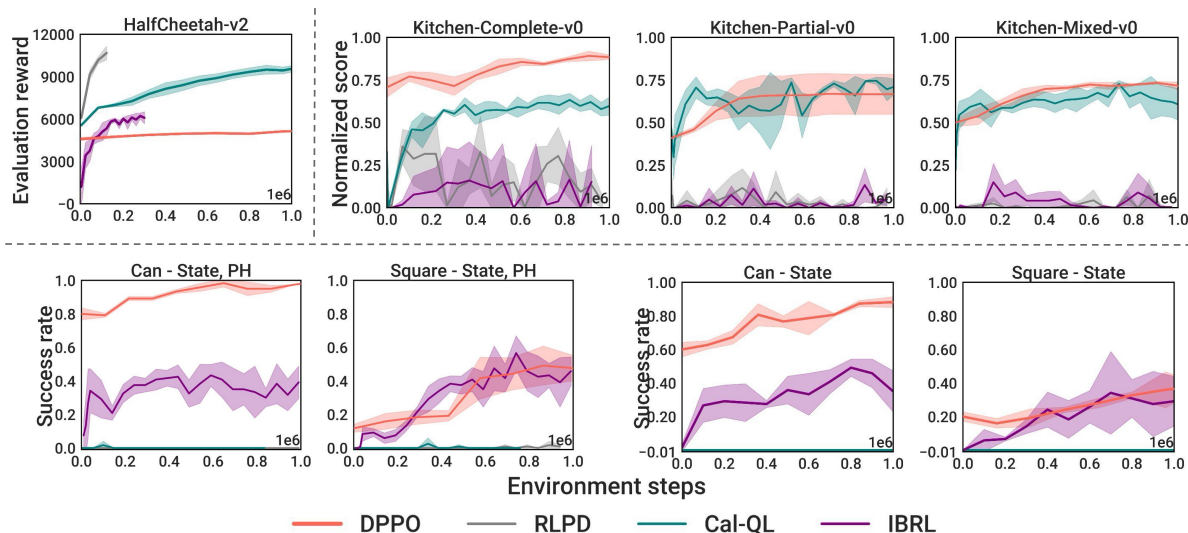


图 6：与其他演示增强强化学习算法的比较。左上：HalfCheetah-v2 任务 [11] 在五个随机种子上的平均值。右上：FRANKA-KITCHEN 任务 [32] 在三个随机种子上的平均值。底行：ROBOMIMIC 任务 [60] 在三个随机种子上的平均值，使用状态观测，采用熟练人类（PH）或多人（MH）数据。

⁶ 离线策略方法（基线）通常无法充分利用并行化采样，因为策略更新频率较低（例如，每 50 个样本更新 50 次，而不是每 1 个样本更新 1 次），性能可能受到影响。

5.3 与其他策略参数化方法的比较

我们将 DPPO 与流行的强化学习策略参数化方法进行比较：具有对角协方差的单峰高斯分布 [91] 和高斯混合模型（GMM [8]），使用多层感知机或变换器 [94]，并使用 PPO 目标进行微调。我们将这些方法与 DPPO-MLP 和 DPPO-UNet 进行比较，后者分别使用多层感知机或 UNet 作为网络骨干。我们在 ROBOMIMIC 的四个任务（提升、抓取、方形、搬运）上使用状态和像素输入进行评估。使用状态输入时，DPPO 使用 20 个去噪步骤进行预训练，然后微调最后 10 个步骤。使用像素输入时，DPPO 使用 100 个去噪步骤进行预训练，然后微调 5 个 DDIM 步骤。

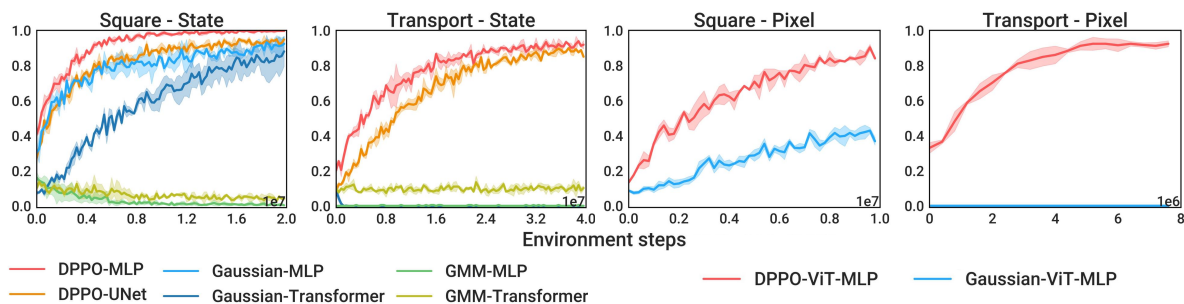


图 7：在 ROBOMIMIC 中更具挑战性的方形和搬运任务上与其他策略参数化方法进行比较，使用状态（左）或像素（右）观测。结果为三个随机种子的平均值。

图 7 展示了更具挑战性的方形和搬运任务的结果——我们将提升和抓取任务的结果推迟到图 A9。使用状态输入时，DPPO 优于高斯和 GMM 策略，在提升和抓取任务中更快收敛到 $\sim 100\%$ 成功率，在方形和具有挑战性的搬运任务上取得更高的最终性能，达到 $> 90\%$ 。UNet 和 MLP 变体表现相似，后者训练速度稍快。使用像素输入时，我们采用 Hu 等人提出的基于视觉变换器（ViT）的图像编码器 [40] 和 MLP 头，并比较由此产生的变体 DPPO-ViT-MLP 和 Gaussian-ViT-MLP（由于基于状态的训练中 GMM 性能较差，我们省略了 GMM）。虽然两者在提升和抓取任务上表现相当，但 DPPO 在方形任务上训练更快且准确率更高，在搬运任务上大幅超越，而高斯策略未能从其 0% 的预训练成功率中提升。据我们所知，DPPO 是第一个从状态或像素输入中解决搬运任务并达到高（ $>50\%$ ）成功率的强化学习算法。

5.4 在 Furniture-Bench 上的评估及仿真到现实迁移

本文在 FURNITURE-BENCH 的长期操作任务上评估 DPPO [36]。我们将 DPPO 与高斯多层感知机（Gaussian-MLP）进行比较，后者是第 5.3 节中整体最有效的基线。图 8（上排）展示了微调迭代过程中的评估成功率。DPPO 表现出很强的训练稳定性，并在所有六种设置下提升了策略性能。高斯多层感知机在中等随机性的三个任务中（除 Lamp 任务的一个种子外）以及低随机性的圆桌任务中，成功率均降至零。需要注意的是，我们仅使用了 50 条人类演示进行预训练；我们预期 DPPO 可以利用额外的人类数据（更好的状态空间覆盖）在中等随机性任务上进一步提升，这一点在附录 C.3 中关于预训练数据对 DPPO 影响的消融研究中得到了证实。

仿真到现实迁移。我们在物理硬件上零样本（即无真实数据微调/协同训练）评估了在仿真单腿任务中训练的 DPPO 和高斯策略，共进行 20 次试验。两种策略均以 10Hz 运行。更多仿真训练和硬件细节请参见附录 E.8。图 8（下排）展示了预训练和微调后的仿真与硬件成功率。值得注意的是，DPPO 将

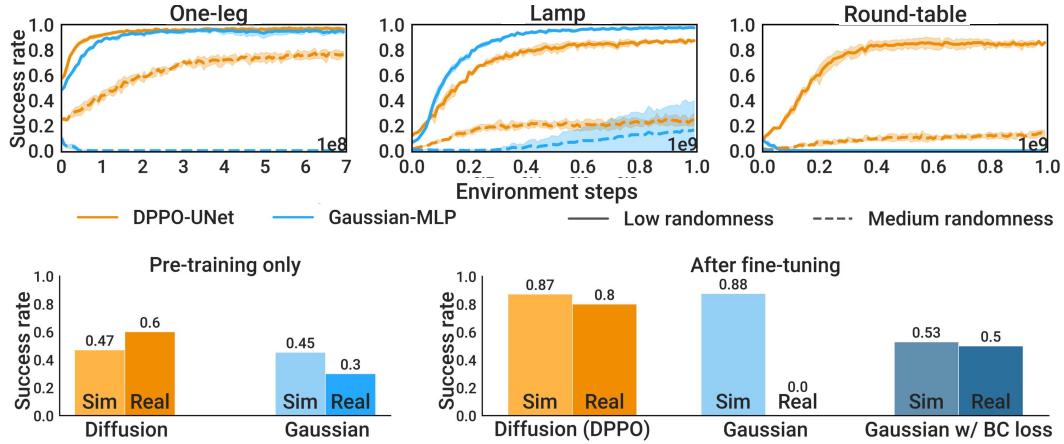


图 8: (上) 仿真 FURNITURE-BENCH 任务中 DPPO 与高斯多层感知机基线的对比。结果为三个种子的平均值。(下) 单腿任务的仿真到现实迁移结果。

真实世界性能提升至 80% (20 次试验中成功 16 次)。尽管高斯策略在微调后的仿真中取得了较高的成功率 (88%)，但在硬件上完全失败 (0%)。补充视频显示其行为不稳定且抖动。为了进行更强的比较，我们还使用辅助行为克隆损失 [93] 对高斯策略进行微调，以鼓励微调后的策略接近基础策略。然而，这限制了微调效果，仅导致仿真成功率为 53%，现实成功率为 50%。定性来看，我们发现微调后的策略比仅预训练的策略更鲁棒，并表现出更多的纠正行为，尤其是在任务的插入阶段；这些行为在

图 2 和图 9 展示了硬件上的代表性运行过程。总体而言，这些结果证明了 DPPO 强大的仿真到现实迁移能力；第 6 节提供了关于这一现象可能原因的推测性机制。

5.5 消融研究总结

我们在附录 C.2 中进行了广泛的消融研究。主要发现包括：(1) 对于具有挑战性的任务，使用依赖于环境状态但独立于去噪动作的价值估计器对性能至关重要；我们推测这与扩散策略的高随机性有关；(2) 在 DPPO 探索中，对去噪噪声水平进行裁剪存在一个最佳点，需要在探索不足和动作噪声过大之间进行权衡；(3) DPPO 对微调少于 K 个去噪步骤具有鲁棒性，从而提高了运行时间并保持了可比的性能；(4) 在不同水平的专家演示数据下，DPPO 相比高斯多层感知机基线取得了改进，并且在 GYM 环境中从头开始训练时，达到了可比的最终性能和样本效率。

6 理解 DPPO 的性能

DPPO 相对于第 5.3 节中流行的高斯和高斯混合模型方法的改进最初令人惊讶，因为 DPPO 求解的扩散策略马尔可夫决策过程 (第 4 节) 比其他方法求解的原始环境马尔可夫决策过程要长得多。这促使我们研究导致 DPPO 性能提升的因素。我们的发现突出了三个主要贡献因素：

- (1) DPPO 在预训练数据流形 [26] 附近诱导结构化探索。
- (2) DPPO 通过多步去噪过程逐步更新动作分布，这可以灵活且鲁棒地防止策略崩溃。



图 9：硬件评估中预训练与微调 DPPO 策略的定性比较。（A）预训练策略的成功执行。（B）由于插入不精确导致预训练策略执行失败。（C）微调策略的成功执行。（D）微调策略的成功执行，表现出纠正行为。

（3）DPPO 使得微调后的策略对动力学和初始状态分布的扰动具有鲁棒性。

我们使用来自 D3IL 基准（benchmark）[46] 的 Avoid 环境，其中机械臂需要到达桌子的另一侧，同时避开一组障碍物（图 10，左上）。动作空间是末端执行器的二维目标位置。D3IL 提供了一组专家演示，覆盖了通往目标线的不同可能路径——我们考虑图 10 中演示的三个子集 M1、M2 和 M3，每个子集具有两种不同的模式。我们选择这些相对简单的设置，每个设置只有两种模式，以便高斯（带探索噪声）⁷ 和 GMM 能够合理地拟合专家数据分布，从而在微调中进行公平比较。我们使用演示对基于 MLP 的扩散、高斯和 GMM 策略（动作块大小 $T_a = 4$ ，除非另有说明）进行预训练。对于微调，当机器人从最上方模式到达目标线时，我们分配稀疏奖励。高斯和 GMM 策略也使用 PPO 目标进行微调。

优势 1：结构化、在流形上的探索。

图 10（右）显示了微调第一次迭代期间 DPPO、高斯和 GMM 的采样轨迹（带探索噪声）。DPPO 在专家数据流形周围进行广泛覆盖的探索，而高斯产生的探索结构较少

⁷ 没有噪声时，高斯策略是完全确定性的，无法捕捉两种模式。

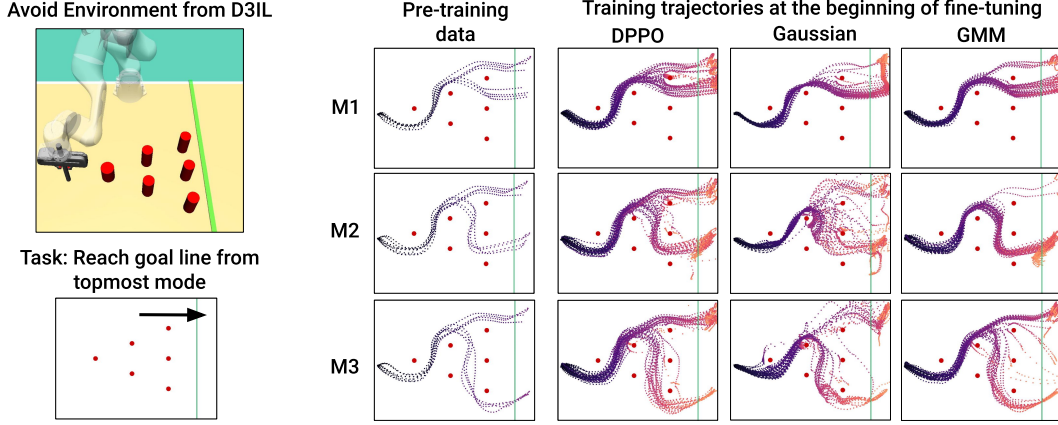


图 10: (左) 我们使用来自 D3IL 基准 [46] 的 Avoid 环境来可视化 DPPO 的探索倾向。我们设计任务为始终从最上方模式到达绿色目标线。(右) 结构化探索。我们展示了在预训练于三组专家演示 M1、M2 和 M3 之后，DPPO、高斯和 GMM 在微调第一次迭代时的采样轨迹。

噪声（尤其是在 M2 中）且 GMM 表现出更窄的覆盖范围。与仅在最终采样动作上添加噪声的高斯策略不同，扩散通过去噪添加多轮噪声。每个去噪步骤在扩展覆盖范围的同时，也将新去噪的动作推向专家数据流形 [69]。此外，扩散参数化与动作块去噪的结合意味着 DPPO 中的策略随机性在动作维度和时间范围上都具有结构性。定量来看，附录中的图 A8 显示，在微调足够数量的去噪步骤的情况下，DPPO 在所有设置中均比高斯和 GMM 获得更高的微调效率，这与第 5.3 节和第 5.5 节的发现一致。然而，当需要激进、非结构化的探索时，我们观察到的 DPPO 在流形上的探索可能会阻碍微调。我们推测在低随机性的 Lamp 环境中就是这种情况，其中 DPPO 略逊于高斯基线（图 8）。此外，我们未观察到 DPPO 在从零开始探索时比高斯显著更好（也不更差）（见附录 C）。因此，我们预期当预训练为相关成功模式提供足够覆盖时，这种结构化的流形上探索能带来最大益处。特别是，我们认为这使得 DPPO 特别适合微调在多任务上预训练的大型扩散策略 [99]，因为给定训练数据的多样性，它可能表现出足够的模式覆盖。

益处 2：多步去噪过程带来的训练稳定性。在图 11（左）中，我们在使用 M2 预训练后进行微调，并尝试通过在微调过程中逐渐向动作添加噪声来破坏微调稳定性（详见附录 E.9）。我们发现高斯和 GMM 的性能均崩溃，而使用 DPPO 时，如果至少使用四个去噪步骤，性能对噪声具有鲁棒性。这一特性还允许 DPPO 对采样动作施加显著噪声，模拟不完美的低级控制器，以促进第 5.4 节中的仿真到现实迁移。在图 11（右）中，我们还发现 DPPO 在微调长动作块时具有更高的训练稳定性，例如最多 $T_a = 16$ ，而高斯和 GMM 可能完全无法改进。

图 12 展示了 DPPO 如何影响多步去噪过程。在微调迭代过程中，动作分布通过去噪步骤逐渐收敛——迭代细化在很大程度上得以保留，而不是例如在第一个微调去噪步骤或最后一个步骤“坍缩”到最优动作。我们推测这有助于 DPPO 的训练稳定性。

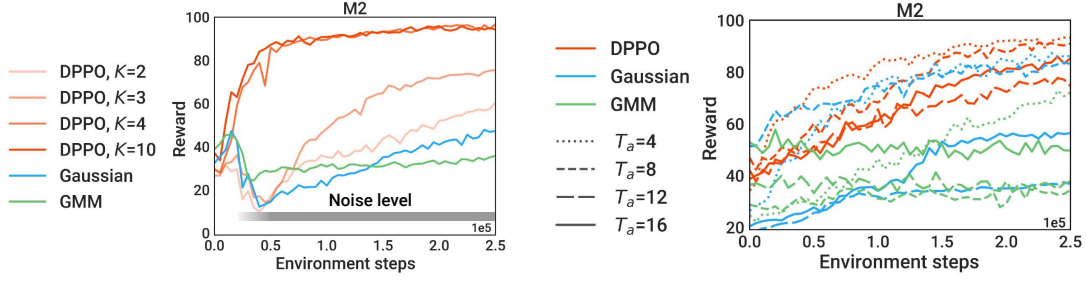


图 11：训练稳定性。使用 M2 预训练后的微调性能（五个种子的平均值，未显示标准差）。（左）在几次训练迭代后向应用的动作中注入噪声。（右）动作块大小 T_a 被改变。

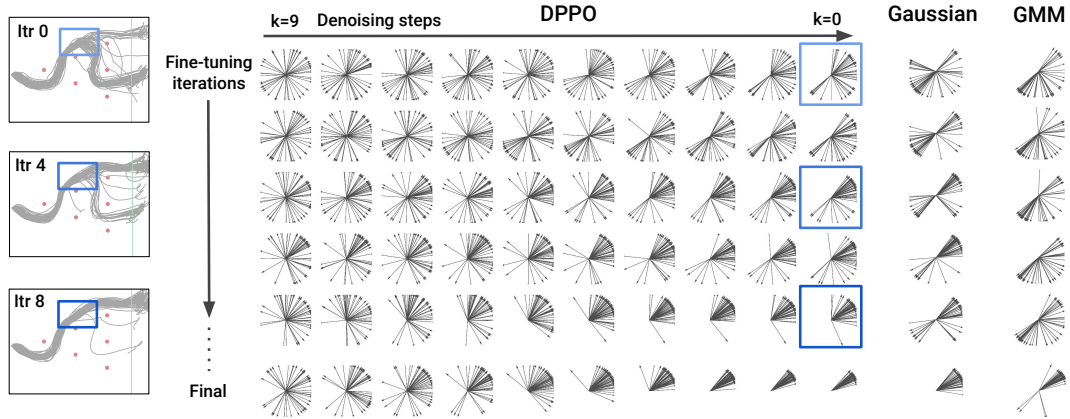


图 12：保留迭代细化。使用 M2 预训练后，在分支点通过微调迭代的 50 条轨迹的二维动作。对于 DPPO，我们还可可视化了每个微调迭代中通过最终去噪步骤的动作分布。

优势 3：鲁棒且可泛化的微调策略。DPPO 还生成了对动力学和初始状态分布扰动具有鲁棒性的最终策略。在图 13 中，我们再次向从微调策略采样的动作中添加噪声（训练期间未应用噪声），发现与高斯策略相比，DPPO 策略对噪声表现出很强的鲁棒性。DPPO 策略还从更大的初始状态分布收敛到（近）最优路径。这一发现呼应了理论保证，即扩散策略能够表示复杂的多模态数据分布，可以有效地从噪声状态中解卷积噪声 [10]，这一特性在 Chen 等人 [15] 中被用于稳定长时程视频生成。

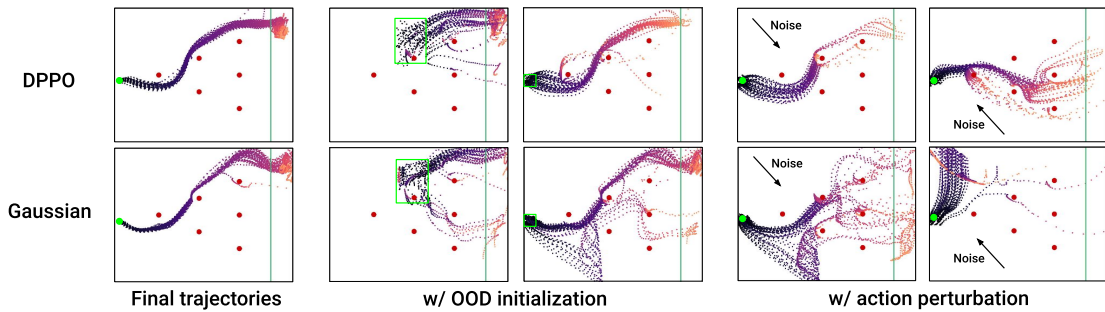


图 13：微调后的策略鲁棒性。绿色圆点/方框表示初始状态区域。

7 结论与未来工作

我们提出扩散策略策略优化 (DPPO)，用于通过策略梯度方法微调预训练的扩散策略。DPPO 利用扩散去噪过程的序列性质，并微调整个扩散马尔可夫决策过程链。DPPO 展现出结构化的在线探索、强大的训练稳定性以及在部署时的鲁棒性和泛化能力。我们在各种强化学习和机器人基准测试中展示了 DPPO 微调的效率和有效性，以及在长时程、多阶段操作任务中 DPPO 策略的强仿真到现实迁移。我们相信 DPPO 将成为预训练加微调流程中训练通用现实世界机器人策略的重要组成部分。为此，我们希望在未来的工作中进一步展示 DPPO 在仿真到现实迁移中的潜力 [18, 19, 52, 77]，其中我们微调一个已在多种不同任务上预训练的基于视觉的策略。我们期望这种预训练能提供一个大而多样的专家数据流形，正如我们在第 6 节中所展示的，DPPO 非常适合利用这一点在微调期间进行更好的探索。我们也期待了解 DPPO 如何与其他决策工具（如基于模型的规划 [45] 和视频预测辅助决策 [15]）结合；这些工具可能有助于解决 DPPO 的主要局限性——其样本效率低于离策略方法——并解锁在物理硬件或生成式仿真（例如基于视频的 [13, 104]）中执行实用的强化学习。最后，正如引言中所指出的，我们热切期待 DPPO 在机器人学之外领域的应用，其中扩散模型已在组合搜索和序列建模中展现出潜力 [56, 73]。

致谢

我们感谢 Lirui Wang 和 Terry Suh 在项目早期阶段的有益讨论。作者部分得到了丰田研究所 (TRI) 的支持。本文仅反映作者的意见和结论，不代表 TRI 或任何其他丰田实体。

参考文献

- [1] J. Achiam. Spinning Up in Deep Reinforcement Learning. 2018.
- [2] A. Ajay, Y. Du, A. Gupta, J. B. Tenenbaum, T. S. Jaakkola, and P. Agrawal. Is conditional generative modeling all you need for decision making? In *The Eleventh International Conference on Learning Representations*, 2023.
- [3] M. Alakuijala, G. Dulac-Arnold, J. Mairal, J. Ponce, and C. Schmid. Residual reinforcement learning from demonstrations. *arXiv preprint arXiv:2106.08050*, 2021.
- [4] O. M. Andrychowicz, B. Baker, M. Chociej, R. Jozefowicz, B. McGrew, J. Pachocki, A. Petron, M. Plappert, G. Powell, A. Ray, et al. Learning dexterous in-hand manipulation. *The International Journal of Robotics Research*, 2020.
- [5] L. Ankile, A. Simeonov, I. Shenfeld, and P. Agrawal. Juicer: Data-efficient imitation learning for robotic assembly. *arXiv*, 2024.
- [6] L. Ankile, A. Simeonov, I. Shenfeld, M. Torne, and P. Agrawal. From imitation to refinement-residual rl for precise visual assembly. *arXiv preprint arXiv:2407.16677*, 2024.
- [7] P. J. Ball, L. Smith, I. Kostrikov, and S. Levine. Efficient online reinforcement learning with offline data. In *International Conference on Machine Learning*, pages 1577–1594. PMLR, 2023.
- [8] C. M. Bishop and N. M. Nasrabadi. *Pattern recognition and machine learning*. Springer, 2006.

- [9] K. Black, M. Janner, Y. Du, I. Kostrikov, and S. Levine. Training diffusion models with reinforcement learning. *arXiv preprint arXiv:2305.13301*, 2023.
- [10] A. Block, A. Jadbabaie, D. Pfrommer, M. Simchowitz, and R. Tedrake. Provable guarantees for generative behavior cloning: Bridging low-level stability and high-level behavior. *Advances in Neural Information Processing Systems*, 2024.
- [11] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.
- [12] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 2020.
- [13] J. Bruce, M. D. Dennis, A. Edwards, J. Parker-Holder, Y. Shi, E. Hughes, M. Lai, A. Mavalankar, R. Steigerwald, C. Apps, et al. Genie: Generative interactive environments. In *Forty-first International Conference on Machine Learning*, 2024.
- [14] S. H. Chan. Tutorial on diffusion models for imaging and vision. *arXiv preprint arXiv:2403.18103*, 2024.
- [15] B. Chen, D. M. Monso, Y. Du, M. Simchowitz, R. Tedrake, and V. Sitzmann. Diffusion forcing: Next-token prediction meets full-sequence diffusion. *arXiv preprint arXiv:2407.01392*, 2024.
- [16] H. Chen, C. Lu, C. Ying, H. Su, and J. Zhu. Offline reinforcement learning via high-fidelity generative behavior modeling. *arXiv preprint arXiv:2209.14548*, 2022.
- [17] T. Chen, J. Xu, and P. Agrawal. A system for general in-hand object re-orientation. In *Conference on Robot Learning*, 2022.
- [18] Y. Chen, C. Wang, L. Fei-Fei, and C. K. Liu. Sequential dexterity: Chaining dexterous policies for long-horizon manipulation. *arXiv preprint arXiv:2309.00987*, 2023.
- [19] C. Chi, B. Burchfiel, E. Cousineau, S. Feng, and S. Song. Iterative residual policy: for goal-conditioned dynamic manipulation of deformable objects. *The International Journal of Robotics Research*, 2024.
- [20] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 2024.
- [21] K. Clark, P. Vicol, K. Swersky, and D. J. Fleet. Directly fine-tuning diffusion models on differentiable rewards. *arXiv preprint arXiv:2309.17400*, 2023.
- [22] Z. Ding and C. Jin. Consistency models as a rich and efficient policy class for reinforcement learning. *arXiv preprint arXiv:2309.16984*, 2023.
- [23] L. Engstrom, A. Ilyas, S. Santurkar, D. Tsipras, F. Janoos, L. Rudolph, and A. Madry. Implementation matters in deep rl: A case study on ppo and trpo. In *International conference on learning representations*, 2019.
- [24] Y. Fan and K. Lee. Optimizing ddpm sampling with shortcut fine-tuning. *arXiv preprint arXiv:2301.13362*, 2023.
- [25] Y. Fan, O. Watkins, Y. Du, H. Liu, M. Ryu, C. Boutilier, P. Abbeel, M. Ghavamzadeh, K. Lee, and K. Lee. Reinforcement learning for fine-tuning text-to-image diffusion models. *Advances in Neural Information Processing Systems*, 2024.
- [26] C. Fefferman, S. Mitter, and H. Narayanan. Testing the manifold hypothesis. *Journal of the American Mathematical Society*, 2016.
- [27] P. Florence, L. Manuelli, and R. Tedrake. Self-supervised correspondence in visuomotor policy learning. *IEEE Robotics and Automation Letters*, 2019.

- [28] P. Florence, C. Lynch, A. Zeng, O. A. Ramirez, A. Wahid, L. Downs, A. Wong, J. Lee, I. Mordatch, and J. Tompson. Implicit behavioral cloning. In *Conference on Robot Learning*. PMLR, 2022.
- [29] J. Fu, A. Kumar, O. Nachum, G. Tucker, and S. Levine. D4rl: Datasets for deep data-driven reinforcement learning. *arXiv preprint arXiv:2004.07219*, 2020.
- [30] Z. Fu, T. Z. Zhao, and C. Finn. Mobile aloha: Learning bimanual mobile manipulation with low-cost whole-body teleoperation. *arXiv preprint arXiv:2401.02117*, 2024.
- [31] W. Goo and S. Niekum. Know your boundaries: The necessity of explicit behavioral cloning in offline rl. *arXiv preprint arXiv:2206.00695*, 2022.
- [32] A. Gupta, V. Kumar, C. Lynch, S. Levine, and K. Hausman. Relay policy learning: Solving long-horizon tasks via imitation and reinforcement learning. *arXiv preprint arXiv:1910.11956*, 2019.
- [33] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pages 1861–1870. PMLR, 2018.
- [34] S. Haldar, J. Pari, A. Rai, and L. Pinto. Teach a robot to fish: Versatile imitation from one minute of demonstrations. *arXiv preprint arXiv:2303.01497*, 2023.
- [35] P. Hansen-Estruch, I. Kostrikov, M. Janner, J. G. Kuba, and S. Levine. Idql: Implicit q-learning as an actor-critic method with diffusion policies. *arXiv preprint arXiv:2304.10573*, 2023.
- [36] M. Heo, Y. Lee, D. Lee, and J. J. Lim. Furniturebench: Reproducible real-world benchmark for long-horizon complex manipulation. *arXiv preprint arXiv:2305.12821*, 2023.
- [37] T. Hester, M. Vecerik, O. Pietquin, M. Lanctot, T. Schaul, B. Piot, D. Horgan, J. Quan, A. Sendonaris, I. Osband, et al. Deep q-learning from demonstrations. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.
- [38] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 2020.
- [39] J. Ho, W. Chan, C. Saharia, J. Whang, R. Gao, A. Gritsenko, D. P. Kingma, B. Poole, M. Norouzi, D. J. Fleet, et al. Imagen video: High definition video generation with diffusion models. *arXiv preprint arXiv:2210.02303*, 2022.
- [40] H. Hu, S. Mirchandani, and D. Sadigh. Imitation bootstrapped reinforcement learning. *arXiv preprint arXiv:2311.02198*, 2023.
- [41] S. Huang, R. F. J. Dossa, C. Ye, J. Braga, D. Chakraborty, K. Mehta, and J. G. AraŹsjo. Cleanrl: High-quality single-file implementations of deep reinforcement learning algorithms. *Journal of Machine Learning Research*, 2022.
- [42] Z. Huang, L. Yang, X. Zhou, Z. Zhang, W. Zhang, X. Zheng, J. Chen, Y. Wang, C. Bin, and W. Yang. Protein-ligand interaction prior for binding-aware 3d molecule diffusion models. In *The Twelfth International Conference on Learning Representations*, 2024.
- [43] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter. Learning agile and dynamic motor skills for legged robots. *Science Robotics*, 2019.
- [44] M. T. Jackson, M. T. Matthews, C. Lu, B. Ellis, S. Whiteson, and J. Foerster. Policy-guided diffusion. *arXiv preprint arXiv:2404.06356*, 2024.
- [45] M. Janner, Y. Du, J. B. Tenenbaum, and S. Levine. Planning with diffusion for flexible behavior synthesis. *arXiv preprint arXiv:2205.09991*, 2022.
- [46] X. Jia, D. Blessing, X. Jiang, M. Reuss, A. Donat, R. Lioutikov, and G. Neumann. Towards diverse behaviors: A benchmark for imitation learning with human demonstrations. *arXiv preprint arXiv:2402.14606*, 2024.

- [47] B. Kang, X. Ma, C. Du, T. Pang, and S. Yan. Efficient diffusion policies for offline reinforcement learning. *Advances in Neural Information Processing Systems*, 2024.
- [48] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza. Champion-level drone racing using deep reinforcement learning. *Nature*, 2023.
- [49] Z. Kong, W. Ping, J. Huang, K. Zhao, and B. Catanzaro. Diffwave: A versatile diffusion model for audio synthesis. *arXiv preprint arXiv:2009.09761*, 2020.
- [50] S. Lee, Y. Wang, H. Etukuru, H. J. Kim, N. M. M. Shafiullah, and L. Pinto. Behavior generation with latent actions. *arXiv preprint arXiv:2403.03181*, 2024.
- [51] K. Lei, Z. He, C. Lu, K. Hu, Y. Gao, and H. Xu. Uni-o4: Unifying online and offline deep reinforcement learning with multi-step on-policy optimization. *arXiv preprint arXiv:2311.03351*, 2023.
- [52] J. Liang, S. Saxena, and O. Kroemer. Learning active task-oriented exploration policies for bridging the sim-to-real gap. *arXiv preprint arXiv:2006.01952*, 2020.
- [53] Z. Liang, Y. Mu, M. Ding, F. Ni, M. Tomizuka, and P. Luo. Adaptdiffuser: Diffusion models as adaptive self-evolving planners. *arXiv preprint arXiv:2302.01877*, 2023.
- [54] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.
- [55] Y. Lin, A. S. Wang, G. Sutanto, A. Rai, and F. Meier. Polymetis. <https://facebookresearch.github.io/fairo/polymetis/>, 2021.
- [56] A. Lou, C. Meng, and S. Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. *stat*, 2024.
- [57] J. Luo, Z. Hu, C. Xu, Y. L. Tan, J. Berg, A. Sharma, S. Schaal, C. Finn, A. Gupta, and S. Levine. Serl: A software suite for sample-efficient robotic reinforcement learning. *arXiv preprint arXiv:2401.16013*, 2024.
- [58] S. Luo, Y. Su, X. Peng, S. Wang, J. Peng, and J. Ma. Antigen-specific antibody design and optimization with diffusion-based generative models for protein structures. *Advances in Neural Information Processing Systems*, 2022.
- [59] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, et al. Isaac gym: High performance gpu-based physics simulation for robot learning. *arXiv preprint arXiv:2108.10470*, 2021.
- [60] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. In *arXiv preprint arXiv:2108.03298*, 2021.
- [61] A. Nair, A. Gupta, M. Dalal, and S. Levine. Awac: Accelerating online reinforcement learning with offline datasets. *arXiv preprint arXiv:2006.09359*, 2020.
- [62] M. Nakamoto, S. Zhai, A. Singh, M. Sobol Mark, Y. Ma, C. Finn, A. Kumar, and S. Levine. Cal-ql: Calibrated offline rl pre-training for efficient online fine-tuning. *Advances in Neural Information Processing Systems*, 36, 2024.
- [63] A. Q. Nichol and P. Dhariwal. Improved denoising diffusion probabilistic models. In *International conference on machine learning*, 2021.
- [64] T. Osa, J. Pajarinen, G. Neumann, J. A. Bagnell, P. Abbeel, J. Peters, et al. An algorithmic perspective on imitation learning. *Foundations and Trends® in Robotics*, 2018.
- [65] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 2022.

- [66] T. Pearce, T. Rashid, A. Kanervisto, D. Bignell, M. Sun, R. Georgescu, S. V. Macua, S. Z. Tan, I. Momennejad, K. Hofmann, et al. Imitating human behaviour with diffusion models. *arXiv preprint arXiv:2301.10677*, 2023.
- [67] X. B. Peng, A. Kumar, G. Zhang, and S. Levine. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning. *arXiv preprint arXiv:1910.00177*, 2019.
- [68] X. B. Peng, Z. Ma, P. Abbeel, S. Levine, and A. Kanazawa. Amp: Adversarial motion priors for stylized physics-based character control. *ACM Transactions on Graphics (ToG)*, 2021.
- [69] F. Permenter and C. Yuan. Interpreting and improving diffusion models from an optimization perspective. *arXiv preprint arXiv:2306.04848*, 2023.
- [70] J. Peters and S. Schaal. Reinforcement learning by reward-weighted regression for operational space control. In *Proceedings of the 24th international conference on Machine learning*, 2007.
- [71] D. A. Pomerleau. Alvin: An autonomous land vehicle in a neural network. *Advances in neural information processing systems*, 1988.
- [72] B. Poole, A. Jain, J. T. Barron, and B. Mildenhall. Dreamfusion: Text-to-3d using 2d diffusion. *arXiv preprint arXiv:2209.14988*, 2022.
- [73] M. Popova, O. Isayev, and A. Tropsha. Deep reinforcement learning for de novo drug design. *Science advances*, 2018.
- [74] M. Psenka, A. Escontrela, P. Abbeel, and Y. Ma. Learning a diffusion model policy from rewards via q-score matching. *arXiv preprint arXiv:2312.11752*, 2023.
- [75] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, 2021.
- [76] A. Rajeswaran, V. Kumar, A. Gupta, G. Vezzani, J. Schulman, E. Todorov, and S. Levine. Learning complex dexterous manipulation with deep reinforcement learning and demonstrations. *arXiv preprint arXiv:1709.10087*, 2017.
- [77] A. Z. Ren, H. Dai, B. Burchfiel, and A. Majumdar. AdaptSim: Task-driven simulation adaptation for sim-to-real transfer. In *Proceedings of the Conference on Robot Learning (CoRL)*, 2023.
- [78] M. Reuss, M. Li, X. Jia, and R. Lioutikov. Goal-conditioned imitation learning using score-based diffusion policies. *arXiv preprint arXiv:2304.02532*, 2023.
- [79] M. Rigter, J. Yamada, and I. Posner. World models via policy-guided trajectory diffusion. *arXiv preprint arXiv:2312.08533*, 2023.
- [80] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022.
- [81] O. Ronneberger, P. Fischer, and T. Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical image computing and computer-assisted intervention (MICCAI)*, 2015.
- [82] N. Ruiz, Y. Li, V. Jampani, Y. Pritch, M. Rubinstein, and K. Aberman. Dreambooth: Fine tuning text-to-image diffusion models for subject-driven generation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2023.
- [83] S. S. Sahoo, M. Arriola, Y. Schiff, A. Gokaslan, E. Marroquin, J. T. Chiu, A. Rush, and V. Kuleshov. Simple and effective masked diffusion language models. *arXiv preprint arXiv:2406.07524*, 2024.
- [84] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- [85] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

- [86] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International conference on machine learning*, 2015.
- [87] J. Song, C. Meng, and S. Ermon. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*, 2020.
- [88] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020.
- [89] A. Sridhar, D. Shah, C. Glossop, and S. Levine. Nomad: Goal masked diffusion policies for navigation and exploration. *arXiv preprint arXiv:2310.07896*, 2023.
- [90] R. S. Sutton and A. G. Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
- [91] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in neural information processing systems*, 1999.
- [92] E. Todorov, T. Erez, and Y. Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ international conference on intelligent robots and systems*, 2012.
- [93] M. Torne, A. Simeonov, Z. Li, A. Chan, T. Chen, A. Gupta, and P. Agrawal. Reconciling reality through simulation: A real-to-sim-to-real approach for robust manipulation. *arXiv preprint arXiv:2403.03949*, 2024.
- [94] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 2017.
- [95] M. Vecerik, T. Hester, J. Scholz, F. Wang, O. Pietquin, B. Piot, N. Heess, T. Rothörl, T. Lampe, and M. Riedmiller. Leveraging demonstrations for deep reinforcement learning on robotics problems with sparse rewards. *arXiv preprint arXiv:1707.08817*, 2017.
- [96] S. Venkatraman, S. Khaitan, R. T. Akella, J. Dolan, J. Schneider, and G. Berseth. Reasoning with latent diffusion in offline reinforcement learning. *arXiv preprint arXiv:2309.06599*, 2023.
- [97] B. Wallace, M. Dang, R. Rafailov, L. Zhou, A. Lou, S. Purushwalkam, S. Ermon, C. Xiong, S. Joty, and N. Naik. Diffusion model alignment using direct preference optimization. *arXiv preprint arXiv:2311.12908*, 2023.
- [98] J. Wang and E. Olson. Apriltag 2: Efficient and robust fiducial detection. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2016.
- [99] L. Wang, J. Zhao, Y. Du, E. H. Adelson, and R. Tedrake. Poco: Policy composition from and for heterogeneous robot learning. *arXiv preprint arXiv:2402.02511*, 2024.
- [100] Z. Wang, J. J. Hunt, and M. Zhou. Diffusion policies as an expressive policy class for offline reinforcement learning. *arXiv preprint arXiv:2208.06193*, 2022.
- [101] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 1992.
- [102] J. Yang, M. S. Mark, B. Vu, A. Sharma, J. Bohg, and C. Finn. Robot fine-tuning made easy: Pre-training rewards and policies for autonomous real-world reinforcement learning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4804–4811. IEEE, 2024.
- [103] L. Yang, Z. Huang, F. Lei, Y. Zhong, Y. Yang, C. Fang, S. Wen, B. Zhou, and Z. Lin. Policy representation via diffusion probability model for reinforcement learning. *arXiv preprint arXiv:2305.13122*, 2023.
- [104] M. Yang, Y. Du, K. Ghasemipour, J. Tompson, D. Schuurmans, and P. Abbeel. Learning interactive real-world simulators. *arXiv preprint arXiv:2310.06114*, 2023.
- [105] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu. 3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations. In *ICRA 2024 Workshop on 3D Visual Representations for Robot Manipulation*.

- [106] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.
- [107] H. Zhu, A. Gupta, A. Rajeswaran, S. Levine, and V. Kumar. Dexterous manipulation with deep reinforcement learning: Efficient, general, and low-cost. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 3651–3657. IEEE, 2019.
- [108] Z. Zhu, H. Zhao, H. He, Y. Zhong, S. Zhang, Y. Yu, and W. Zhang. Diffusion models for reinforcement learning: A survey. *arXiv preprint arXiv:2311.01223*, 2023.

A 扩展相关工作

A.1 使用离线数据训练机器人策略的强化学习

在此，我们讨论使用强化学习（RL）结合离线数据训练机器人策略的相关工作，以帮助强化学习在稀疏奖励环境中更好地在线探索。一种简单的形式是利用离线数据预训练策略，通常采用行为克隆（Behavior Cloning），然后在线微调策略。这是 DPPO 采用的方法。通常，会应用正则化损失来约束微调后的策略保持接近基础策略，从而产生自然的微调行为，并且往往能改善学习效果 [76, 93, 107]。DPPO 在微调时不应应用正则化，因为我们发现流形上探索有助于 DPPO 在微调后保持自然行为（第 5.4 节）。另一种流行的方法是在冻结的基础策略之上用强化学习学习残差策略 [3, 34]。与我们的工作更接近的是 Ankile 等人 [6] 的方法，该方法在预训练的分块扩散策略之上，使用在线策略强化学习训练单步残差非扩散策略。这种方法具有完全闭环的优点，但缺乏 DPPO 的结构化流形上探索。另一种混合方法来自 Hu 等人 [40]，他们使用预训练和微调的策略来采样在线经验。另一类流行的工作不是使用离线数据训练基础策略，而是直接将数据加入回放缓冲区，用于单阶段的在线策略学习 [37, 61, 95]。Ball 等人 [7] 最近提出的 RLPD 方法，通过结合评论家集成等技术，进一步提高了样本效率。Luo 等人 [57] 展示了 RLPD 解决现实世界操作任务的能力（尽管通常不如 DPPO 解决的任务具有挑战性）。其他方法，包括 **Cal-QL**，建立在离线强化学习之上，从离线数据中学习，然后切换到在线强化学习，同时仍从离线数据中采样 [35, 62, 102]。通常需要解决离线数据与在线策略之间的分布不匹配问题：**Cal-QL** 提出校准保守 Q 学习（Calibrated Conservative Q-Learning），学习一个离线 Q 函数，该函数是所学策略真实值的下界；Lei 等人 [51] 提出在预训练期间使用集成行为克隆以促进策略多样性。

A.2 基于扩散的强化学习方法

本节讨论直接训练或改进基于扩散的策略（Policy）的强化学习（RL）方法。我们在第 5.1 节中比较的基线（Baseline）也在下文讨论，并以相应颜色突出显示。我们还建议读者参考 Zhu 等人 [108] 关于扩散模型（Diffusion Model）用于强化学习的综述。大多数先前工作集中于具有静态数据集的离线设置。一类工作聚焦于状态轨迹规划，并引导去噪采样过程，使采样动作满足某些期望目标。Janner 等人 [45] 应用分类器引导（Classifier Guidance），生成具有更高预测奖励的轨迹。Ajay 等人 [2] 引入无分类器引导（Classifier-Free Guidance），避免学习噪声状态的价值。另一类工作将扩散模型用作动作策略（而非状态规划器），并通常应用 Q 学习（Q-Learning）。DQL[100] 提出扩散 Q 学习（Diffusion Q-Learning），为最终去噪动作学习状态-动作评论家（Critic），并将评论家的梯度通过整个扩散策略（Actor）去噪链反向传播，类似于通常的 Q 学习。IDQL[35]，即隐式扩散 Q 学习（Implicit Diffusion Q-Learning），提出学习评论家以在推理时选择动作用于训练或评估，同时将演员拟合到所有采样动作。Kang 等人 [47] 则提出使用评论家重新加权采样动作以更新演员本身，类似于我们工作中引入的加权回归基线 DAWR 和 DRWR。Goo 和 Niekum[31] 同样以 AWR[67] 的精神提取策略。Chen 等人 [16] 基于演员的样本使用价值迭代（Value Iteration）训练评论家。最后，Jackson 等人 [44] 探索使用扩散引导将离线数据移向目标轨迹分布。我们注意到，像 DQL 和 IDQL 这样的方法也可以应用于在线设置。少量

本文工作也完全聚焦于在线设定。DIPO [103] 与 DQL 及相关工作的不同之处在于，它使用评论家来更新采样动作（“动作梯度”），而不是演员——随后用回放缓冲区中更新后的动作来拟合演员。QSM，即 Q 分数匹配 [74]，认为优化整条去噪动作链的似然可能效率低下（这与我们在微调设定中的发现相反），并提出通过迭代地将演员的梯度（即分数）与评论家的动作梯度对齐来学习最优策略。Rigter 等人 [79] 提出学习一个扩散动态模型，为在线训练非扩散强化学习策略生成合成轨迹。我们注意到，几乎所有先前基于扩散的强化学习工作（离线或在线）都依赖于近似状态-动作 Q 函数，并以某种形式使用它来更新扩散演员——由于多步去噪过程，策略梯度更新一直被认为具有挑战性 [74, 103]。不准确的 Q 值可能导致对演员的有偏更新，进而导致训练崩溃：如图 5 所示，它从不错的预训练性能开始，但成功率迅速降至零，并且由于稀疏奖励设置，此后也无法恢复。虽然 Q 学习方法在能够解决感兴趣的任务时通常能获得更好的样本效率，但我们的重点主要放在具有挑战性的长时程机器人操作任务上，这些任务非常需要训练稳定性。

与 Psenka 等人 [74] 中策略梯度公式的区别。 Psenka 等人 [74] 第 3 节引入了另一种公式，用于推导扩散策略的策略梯度更新。该推导基于将最终去噪动作的对数似然梯度转换为各个去噪动作的对数似然之和。与 DPPO 不同，该公式未将多步去噪过程视为马尔可夫决策过程（MDP）。在策略梯度更新中，Psenka 等人 [74] 对去噪步骤求和，然后对环境步骤取期望，而 DPPO 的更新（4.2）同时对去噪步骤和环境步骤取期望，可能带来更好的样本效率。此外，Psenka 等人 [74] 未提出对扩散应用 PPO 更新或其他修改，并发现这种原始形式的策略梯度更新效果不佳。我们独立于他们的工作提出了 DPPO，并发现 DPPO 在微调设置中非常有效，同时在从头训练中也具有竞争力（附录 C.3）。

B DPPO 实现的附加细节

伪代码。 DPPO 的伪代码如算法 1 所示。DPPO 以使用行为克隆损失 $\mathcal{L}_{\text{BC}}$ 训练的扩散策略 π_θ 作为输入。然后使用 PPO 风格的损失 [85] 对策略进行微调，并仔细处理去噪过程（第 4 节）。

预训练。 扩散策略 π_θ 使用行为克隆损失 [38] 进行预训练：

$$\mathcal{L}_{\text{BC}}(\theta) = \mathbb{E}^{(s_t, a_t^0) \sim \mathcal{D}_{\text{off}}} [\|\varepsilon_t - \varepsilon_\theta(a_t^0, s_t, k)\|^2], \quad (\text{A1})$$

其中 $\mathcal{D}_{\text{off}}$ 是离线数据集， ε_θ 是策略网络，基于噪声动作预测添加到 a_t^0 的采样噪声。我们使用 Nichol 和 Dhariwal[63] 的余弦噪声调度。

环境步优势估计。 我们在算法 1 中使用广义优势估计（Generalized Advantage Estimation, GAE）[84]，参数为 λ ，进行优势估计。GAE- λ 通过级数近似优势函数

$$\hat{A}_{\bar{l}(t,k=0)}^\lambda = \sum_{l=0}^{\infty} (\gamma\lambda)^l \bar{\delta}_{\bar{l}(t,k=0)}, \quad \text{其中} \quad \bar{\delta}_{\bar{l}(t,k)} = \bar{R}_{\bar{l}(t,k)} + \gamma_{\text{ENV}} V_\phi(\bar{s}_{\bar{l}(t+1,k)}) - V_\phi(\bar{s}_{\bar{l}(t,k)}). \quad (\text{A2})$$

Algorithm 1 DPPO

```

1: Pre-train diffusion policy  $\pi_\theta$  with offline dataset  $\mathcal{D}_{\text{off}}$  using BC loss  $\mathcal{L}_{\text{BC}}(\theta)$  Eq. (A1).
2: Initialize value function  $V_\phi$ .
3: for iteration = 1, 2, ... do
4:   Initialize rollout buffer  $\mathcal{D}_{\text{itr}}$ .
5:    $\pi_{\theta_{\text{old}}} = \pi_\theta$ .
6:   for environment = 1, 2, ...,  $N$  in parallel do
7:     Initialize state  $\bar{s}_{\bar{t}(0,K)} = (s_0, a_0^K)$  in  $\mathcal{M}_{\text{DP}}$ .
8:     for environment step  $t = 1, \dots, T$ , denoising step  $k = K - 1, \dots, 0$  do
9:       Sample the next denoised action  $\bar{a}_{\bar{t}(t,k)} = a_t^k \sim \pi_{\theta_{\text{old}}}$ .
10:      if  $k = 0$  then
11:        Run  $a_t^0$  in the environment and observe  $\bar{R}_{\bar{t}(t,0)}$  and  $\bar{s}_{\bar{t}(t+1,K)}$ .
12:      else
13:        Set  $\bar{R}_{\bar{t}(t,k)} = 0$  and  $\bar{s}_{\bar{t}(t,k-1)} = (s_t, a_t^k)$ .
14:      Add  $(k, \bar{s}_{\bar{t}(t,k)}, \bar{a}_{\bar{t}(t,k)}, \bar{R}_{\bar{t}(t,k)})$  to  $\mathcal{D}_{\text{itr}}$ .
15:      Compute advantage estimates  $A^{\pi_{\theta_{\text{old}}}}(s_{\bar{t}(t,k=0)}, a_{\bar{t}(t,k=0)})$  for  $\mathcal{D}_{\text{itr}}$  using GAE Eq. (A2).
16:      for update = 1, 2, ..., num_update do ▷ Based on replay ratio  $N_\theta$ 
17:        for minibatch = 1, 2, ...,  $B$  do
18:          Sample  $(k, \bar{s}_{\bar{t}(t,k)}, \bar{a}_{\bar{t}(t,k)}, \bar{R}_{\bar{t}(t,k)})$  and  $A^{\pi_{\theta_{\text{old}}}}(s_{\bar{t}(t,k)}, a_{\bar{t}(t,k)})$  from  $\mathcal{D}_{\text{itr}}$ .
19:          Compute denoising-discounted advantage  $\hat{A}_{\bar{t}(t,k)} = \gamma_{\text{DENOISE}}^k A^{\pi_{\theta_{\text{old}}}}(s_{\bar{t}(t,0)}, a_{\bar{t}(t,0)})$ .
20:          Optimize  $\pi_\theta$  using policy gradient loss  $\mathcal{L}_\theta$  Eq. (A3).
21:          Optimize  $V_\phi$  using value loss  $\mathcal{L}_\phi$  Eq. (A4).
22: return converged policy  $\pi_\theta$ .

```

值得注意的是，GAE- λ 在一步时序差分 ($\hat{A}^{\lambda=0} = V_\phi(\bar{s}_{\bar{t}(t,k)}) - V_\phi(\bar{s}_{\bar{t}(t,k-1)})$) 与 $\bar{R}_{\bar{t}(t,k)} + \gamma_{\text{ENV}} V_\phi(\bar{s}_{\bar{t}(t+1,k)}) - V_\phi(\bar{s}_{\bar{t}(t,k)})$ 之间进行插值。关于 GAE 参数选择的更多细节，请读者参阅表 A8；关于优势估计器选择的消融实验，请参阅第 C.2 节。注意，在式 (A2) 中我们仅关注 $k = 0$ ，即最终去噪步骤，为 $k = 0$ （即环境步骤）计算优势，而不为中间去噪步骤计算。我们只需对计算出的优势应用去噪折扣，以便将其应用于每个去噪步骤 k 。

微调。在强化学习微调期间，我们使用裁剪目标更新策略 π_θ ：

$$\mathcal{L}_\theta = \mathbb{E}^{\mathcal{D}_{\text{itr}}} \min \left(\hat{A}^{\pi_{\theta_{\text{old}}}}(\bar{s}_{\bar{t}}, \bar{a}_{\bar{t}}) \frac{\bar{\pi}_\theta(\bar{s}_{\bar{t}}, \bar{a}_{\bar{t}})}{\bar{\pi}_{\theta_{\text{old}}}(\bar{s}_{\bar{t}}, \bar{a}_{\bar{t}})}, \hat{A}^{\pi_{\theta_{\text{old}}}}(\bar{s}_{\bar{t}}, \bar{a}_{\bar{t}}) \text{clip} \left(\frac{\bar{\pi}_\theta(\bar{s}_{\bar{t}}, \bar{a}_{\bar{t}})}{\bar{\pi}_{\theta_{\text{old}}}(\bar{s}_{\bar{t}}, \bar{a}_{\bar{t}})}, 1 - \varepsilon, 1 + \varepsilon \right) \right). \quad (\text{A3})$$

如果我们选择仅微调最后 K' 个去噪步骤，那么我们仅从 $\mathcal{D}_{\text{itr}}$ 中采样这些步骤。最后，我们训练价值函数以预测未来折扣奖励之和（即折扣回报）：

$$\mathcal{L}_\phi = \mathbb{E}^{\mathcal{D}_{\text{itr}}} \left[\left\| \sum_{l=0}^{T-t} \gamma_{\text{ENV}}^l \bar{R}_{\bar{t}(t+l,k)} - V_\phi(s_t) \right\|^2 \right]. \quad (\text{A4})$$

与附录 E.3 中的所有基线类似，我们将 N_θ 和 N_ϕ 分别表示为 DPPO 中演员（扩散策略）和价值评论家的重放比率；在实践中我们始终设置 $N_\theta = N_\phi$ 。与通常的 PPO 实现 [41] 类似，当 π_θ 与 $\pi_{\theta_{\text{old}}}$ 之间的 KL 散度达到 1 时，迭代中的批量更新终止，尽管在实践中我们发现这种情况从未发生。

大批量大小。由于 DPPO 中的梯度更新涉及对环境步骤和去噪步骤的期望，我们使用比高斯策略参数化的 PPO 训练更大的批大小。大致上，我们使用的批大小为高斯训练的批大小乘以微调去噪步骤的数量；在某些情况下（如 ROBOMIMIC），我们还观察到可以使用小得多的批大小（接近高斯训练的批大小），并显著提高样本效率。

梯度裁剪比率。我们发现 PPO 裁剪比率 ε 会显著影响 DPPO（以及高斯和 GMM 策略）的训练稳定性，尤其是在稀疏奖励操作任务中。在实践中，我们发现一个好的指标是裁剪分数（批次中被裁剪的单个样本的比例）达到 10% 到 20%，这能带来最佳训练效率。对于不同任务中的每种方法，我们在 $\{.1, .01, .001\}$ 中调整 ε ，并选择满足裁剪分数目标的最高值。经验上我们还发现，在 DPPO 中为较早的去噪步骤使用更高的 ε 可以进一步提高操作任务中的训练稳定性。记 ε_k 为去噪步骤 k 处的裁剪值，实践中我们设置 $\varepsilon_{k=(K-1)} = 0.1\varepsilon_{k=0}$ ，并在中间 k 之间遵循指数调度。

C 附加实验结果

C.1 与使用扩散策略的演示增强 RL 基线进行比较

在第 5.2 节中，我们将 DPPO 与其他演示增强强化学习方法（即 RLPD、**Cal-QL** 和 IBRL）进行了比较——DPPO 使用扩散策略，而基线方法使用高斯策略。在此，我们尝试为基线方法使用扩散策略，结果如图 A1 所示。我们使用动作块大小 $T_a = 1$ 或 $T_a = 4$ 。我们观察到与图 6 中使用高斯策略相似的结果，即 RLPD 和 **Cal-QL** 完全无法解决任务。我们认为 **Cal-QL** 性能较差的原因在于离线强化学习目标（基于学习状态-动作 Q 函数）使得学习 ROBOMIMIC 任务所需的精确连续动作非常困难，无论策略参数化方式如何，这印证了我们在第 5.1 节中比较 DPPO 与基于 Q 学习的扩散强化学习方法时的原始发现。与使用 SAC 目标和回放缓冲区中的专家数据进行训练的 RLPD 相比，IBRL 利用行为克隆预训练能够更有效地学习基础策略，并将其用于在线数据收集。DPPO 受益于直接微调预训练策略（而不是使用预训练策略的经验训练新策略），在 $1e6$ 步之前实现了与 IBRL 相似或更好的样本效率，并收敛到 $\sim 100\%$ 的成功率，而 IBRL 则饱和在较低水平（未显示）。

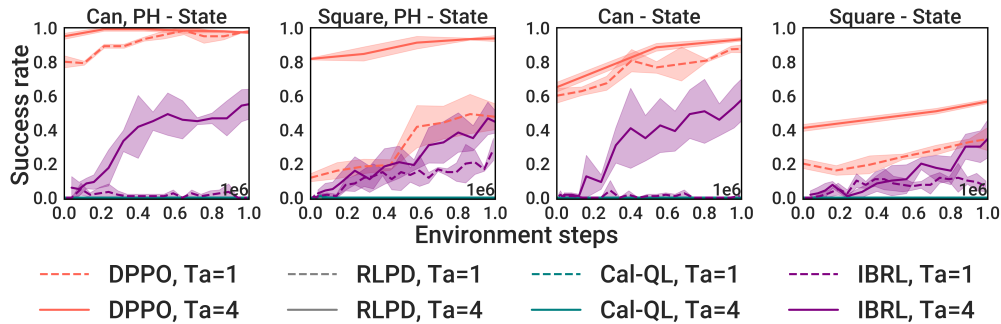


图 A1：为其他演示增强强化学习方法使用扩散策略。结果在三个随机种子上取平均。

C.2 DPPO 设计决策的消融研究

1. 优势估计器的选择。在第 4.3 节中，我们展示了如何通过学习仅依赖于环境状态的 $\tilde{V}(s_t)$ 来高效估计 PPO 更新中使用的优势；DPPO 中使用的优势正式为

$$\hat{A} = \gamma_{\text{DENOISE}}^k (\bar{r}(\bar{s}_t, \bar{a}_t) - \tilde{V}(s_t)).$$

我们现在将这一选择与学习完整状态 $\bar{s}_{t(t,0)}$ 的价值进行比较，该完整状态包括环境状态 s_t 和去噪动作 $a_t^{k=1}$ 。我们还与 Psenka 等人 [74]⁸, $\tilde{Q}(s_t, a_t^{k=0})$ 中使用的状态-动作 Q 函数估计器进行了比较，该估计器在优势中不直接使用 rollout 奖励 $\bar{r}$ 。图 A2 展示了来自 GYM 的 Hopper-v2 和 HalfCheetah-v2 以及来自 ROBOMIMIC 的 Can 和 Square 上的微调结果。在较简单的 Hopper-v2 上，我们观察到两个基线（都估计某个动作的价值）在微调期间获得的奖励高于 DPPO 的选择。然而，在更具挑战性的任务中，DPPO 中使用的仅环境状态优势始终带来最大的性能提升。我们认为，准确估计应用连续且高维动作的价值可能具有挑战性，而基于扩散的策略的高随机性和动作块大小加剧了这一挑战。这里的结果印证了第 5.1 节的发现，即离线 Q 学习方法在 Hopper-v2 和 Walker2D-v2 中表现良好，但在 ROBOMIMIC 的操纵任务中经常出现训练不稳定。

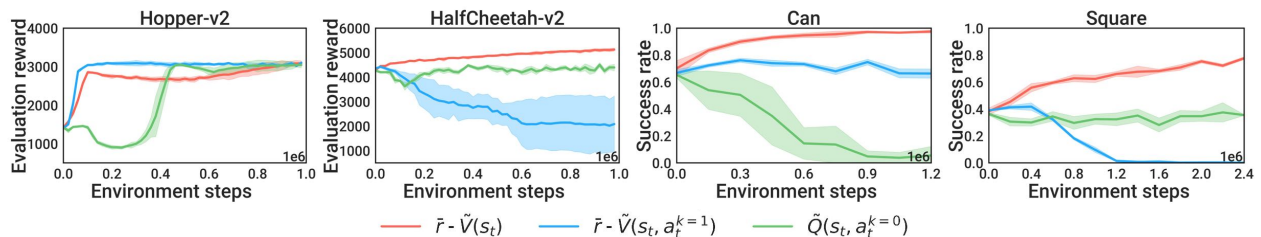


图 A2: 优势估计器的选择。结果在 Hopper-v2 和 HalfCheetah-v2 上对五个种子取平均，在 Can 和 Square 上对三个种子取平均。

去噪折扣因子。我们进一步研究 DPPO 优势估计器中的 γ_{DENOISE} 如何影响微调。使用较小的值（即更多折扣）会降低策略梯度中较早去噪步骤的贡献权重。图 A3 展示了在相同的四个任务中，使用不同 $\gamma_{\text{DENOISE}} \in [0.5, 0.8, 0.9, 1]$ 时的微调结果。我们发现在 Hopper-v2 和 HalfCheetah-v2 中， $\gamma_{\text{DENOISE}} = 0.8$ 带来更好的效率，而较小的 $\gamma_{\text{DENOISE}} = 0.5$ 会减慢训练速度。该值在 Can 中对训练没有明显影响。在 Square 中，较小的 $\gamma_{\text{DENOISE}} = 0.5$ 效果略好。总体而言，在操纵任务中，DPPO 训练似乎对这一选择相对稳健。

2. 扩散噪声调度的选择。如第 4.3 节所述，我们发现将扩散噪声 σ_k 裁剪至较高的最小值 $\sigma_{\min}^{\text{exp}}$ 有助于确保充分探索。在图 A4 中，我们对不同的 $\sigma_{\min}^{\text{exp}} \in \{.001, .01, .1, .2\}$ 进行了分析（保持 $\sigma_{\min}^{\text{prob}} = .1$ 以评估似然）。虽然在 Can 任务中 $\sigma_{\min}^{\text{exp}}$ 的选择不影响微调性能，但在 Square 任务中需要较高的 $\sigma_{\min}^{\text{exp}} = 0.1$ 。Psenka 等人 [74] 采用了离线训练，结合双 Q 学习（根据其开源实现）和去噪步骤上的策略梯度。注意，这是 Psenka 等人 [74] 中的一个基线，被推测为效率低下。我们遵循相同设置，但应用在线策略 PPO 更新。

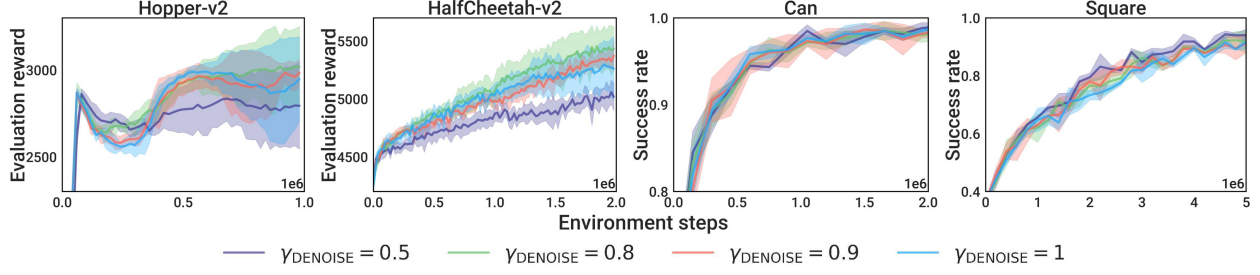


图 A3: 去噪折扣因子的选择。结果在 Hopper-v2 和 HalfCheetah-v2 上取五个种子的平均值，在 Can 和 Square 上取三个种子的平均值。

防止策略崩溃。我们推测这是由于有限的探索导致策略对收集到的样本过度优化，而这些样本的状态-动作覆盖范围有限。我们还可可视化了 D3IL 中 Avoid 任务微调开始时的轨迹。使用较高的 $\sigma_{\min}^{\text{exp}}$ 时，轨迹仍保持在预训练数据的两个模式附近，但在状态空间中表现出更高的覆盖率——我们认为这种额外的覆盖率有助于更好的探索。有趣的是，我们发现提前终止去噪过程也能提供探索噪声并取得可比的结果，但这需要围绕去噪 MDP 进行更复杂的实现。

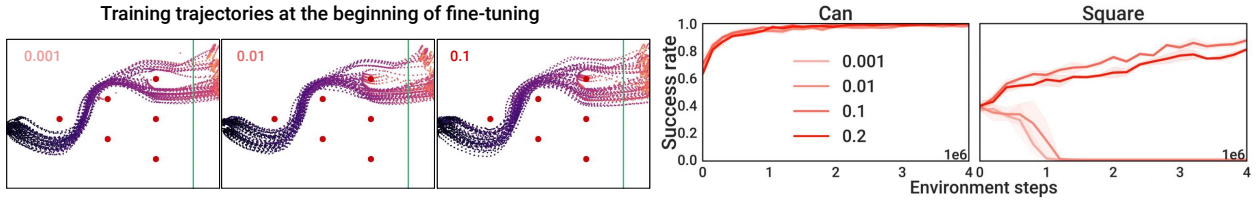


图 A4: 最小扩散噪声的选择。结果取三个种子的平均值。注意在左侧，使用较高的最小噪声水平时，采样轨迹在两个模式处表现出更宽的覆盖率，但仍保持整体结构。

3. 微调去噪步骤数量的选择。我们在图 A5 中考察了 DPPO, K' 中微调去噪步数对微调性能和实际运行时间的影响。我们展示的是各次独立运行（每个 K' 三次）的曲线，而非平均值，因为它们的实际运行时间（X 轴）并不完全对齐。总体而言，微调过少的去噪步数（例如 3 步）会导致次优的渐近性能和较慢的收敛速度，尤其在 Can 任务中更为明显。微调 10 步可获得整体最佳效率。类似结果也出现在图 A8 的 Avoid 任务中。最后，我们注意到 GPU 内存使用量随 K' 线性增长。我们还发现，这里的发现与改变去噪折扣因子 γ_{DENOISE} 的结果大体相关。在策略梯度中对较早的去噪步进行折扣，可以视为对微调去噪步数进行硬限制的一种软版本。根据预训练动作分布所需的微调程度，可以灵活调整 γ_{DENOISE} 和 K' 以达到最佳效率。

C.3 专家数据的影响

我们研究了预训练专家数据量对微调性能的影响。在图 A6 中，我们比较了 DPPO 和 Gaussian 在 Hopper-v2、Square 以及 FURNITURE-BENCH 中的 One-leg 任务上的表现，使用了图中标注的不同数量的专家数据（回合数）。总体而言，我们发现 DPPO 能更好地利用预训练数据，并微调至较高的成功率。值得注意的是，DPPO 仅用 10 个演示回合就在 One-leg 任务上取得了不俗的性能（60% 成功率）。

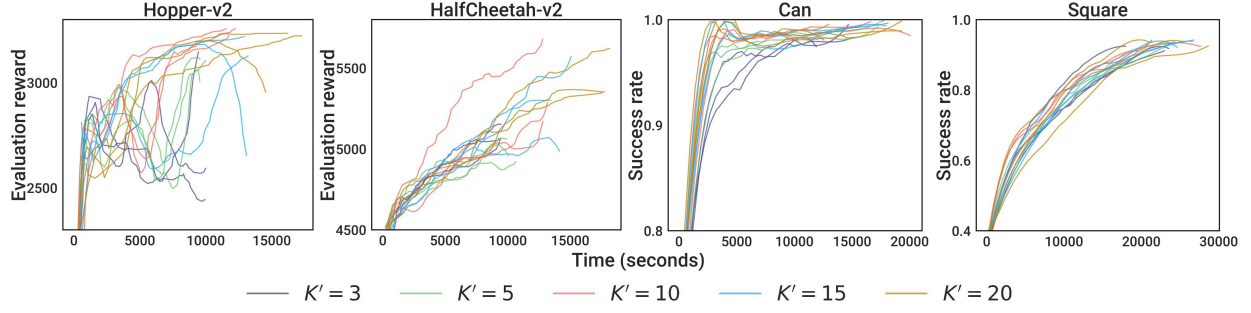


图 A5: 微调去噪步数的选择, K' . 展示了各次独立运行。曲线使用 Savitzky - Golay 滤波器进行了平滑处理。

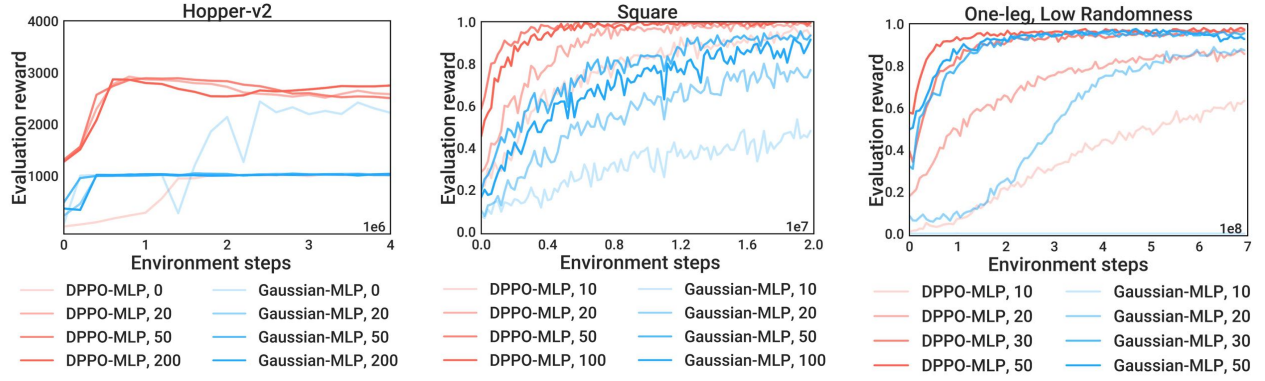


图 A6: 改变专家演示数量。图例中的数字表示预训练中使用的回合数。

从头开始训练。在图 A7 中, 我们比较了 DPPO (10 个去噪步骤) 与从头开始训练的高斯策略 (未在专家数据上预训练) 在三个 OpenAI GYM 任务中的表现。由于使用较大的动作块大小 T_a 会导致从头训练效果不佳 (如图 A6 所示), 我们专注于单动作块 $T_a = 1$ (以及 $T_p = 1$), 这在强化学习基准测试中是典型的做法。尽管我们发现高斯策略的训练速度比 DPPO 更快 (这是预期的, 因为 DPPO 解决的是有效时间范围更长的马尔可夫决策过程), 但 DPPO 仍然达到了合理的最终性能。然而, 由于多步 (10 步) 去噪采样, DPPO 的墙钟时间大约是高斯策略的 $6\times$ 倍。我们希望未来的工作能够探索如何设计去噪步骤的训练课程, 以在训练性能和墙钟效率之间取得最佳平衡。

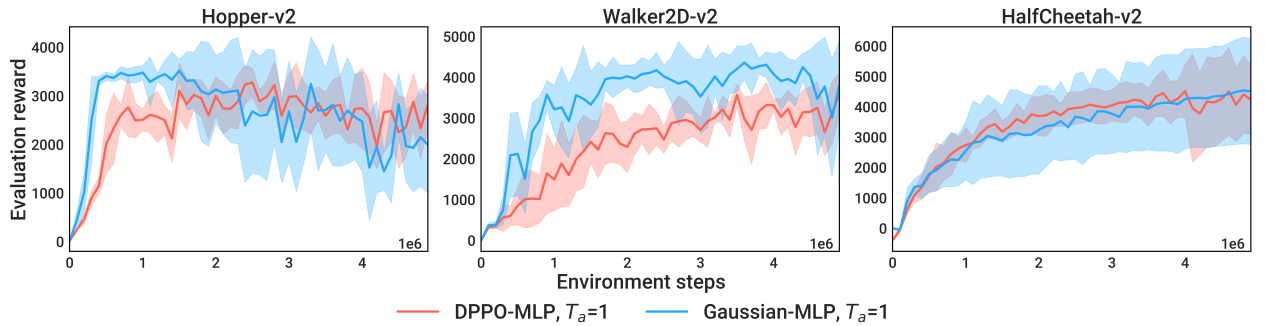


图 A7: 无专家数据/预训练的 GYM 任务。结果在五个随机种子上取平均。

C.4 在 Avoid 任务中与其他策略参数化方法的比较 图 A8 展示了 DPPO 的各种参数化（具有不同数量的微调去噪步骤， K' ）与高斯和 GMM 基线的性能对比。我们研究了 D3IL 中的 Avoid 任务，在按照第 6 节所述使用 M1、M2、M3 的数据进行预训练之后。我们发现，对于 $K' \in \{15, 20\}$ ，DPPO 在所有方法中达到了最高的性能，并且在环境步骤方面训练速度最快；在 M1、M2 上，它似乎也达到了最高的最终性能。 $K' = 10$ 的表现略好于高斯基线，但大致相当，而 GMM 和 $K' < 10$ 的表现则较弱。

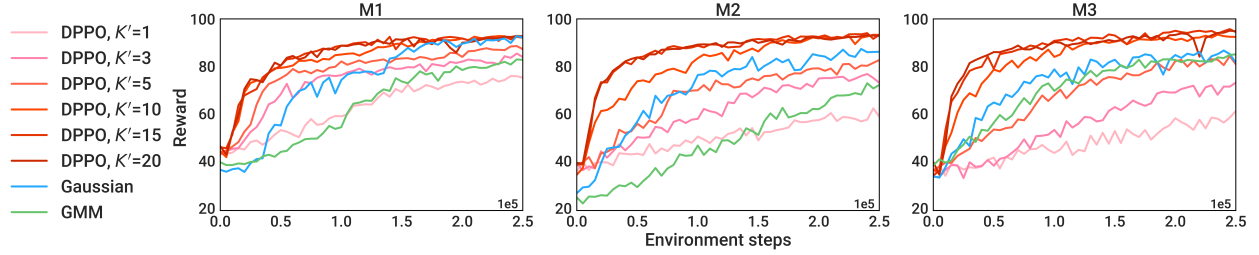


图 A8：在 D3IL 的 Avoid 任务中使用 M1、M2 和 M3 进行预训练后的微调性能（在五个随机种子上取平均，未显示标准差）。比较了 DPPO ($K = 20$ 、高斯和 GMM 策略。我们还扫描了 DPPO 中微调去噪步骤的数量 K' 。

C.5 在 ROBOMIMIC 中较简单任务上与其他策略参数化方法的比较

图 A9 比较了 DPPO 与高斯和 GMM 基线在 ROBOMIMIC 套件中的 Lift 和 Can 环境中，跨多种架构以及状态和像素输入的性能。与 Square 和 Transport（结果见第 5 节）相比，这些环境被认为是“较简单的”，这反映在 DPPO 和高斯基线更高的性能上（GMM 仍然表现不佳）。尽管如此，与高斯基线相比，DPPO 仍然实现了相似甚至更好的样本效率。

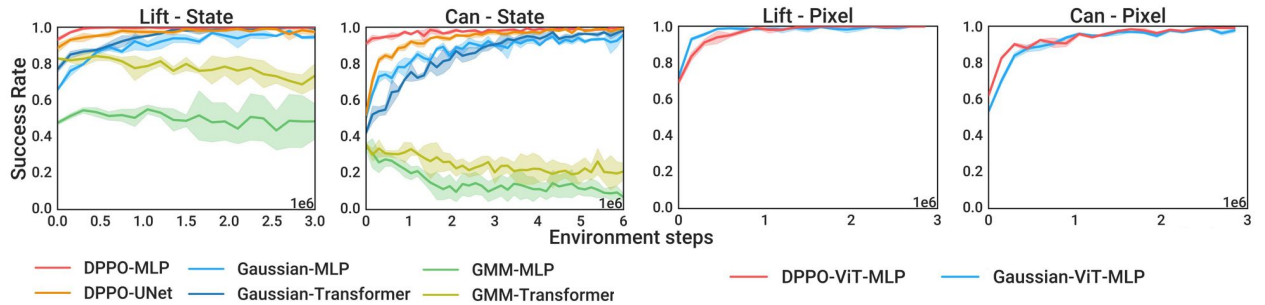


图 A9：在 ROBOMIMIC 中较简单的 Lift 和 Can 任务上，与其他策略参数化方法进行比较，分别使用状态（左）或像素（右）观测。结果为三个随机种子的平均值。

C.6 与使用扩散策略精确似然的策略梯度进行比较

本文实验了另一种新颖的方法（据我们所知，此前尚无工作明确研究过），用于基于扩散的策略执行策略梯度。尽管扩散模型不直接建模动作似然 $p_\theta(a_0|s)$ ，但已有方法可以估计该值，例如通过求解实现 DDPM 的概率流常微分方程 [88]。我们建议读者参考 Song 等人 [88] 的附录 D 以获取全面阐述。我们遵循 Song 等人⁹的官方开源代码，

⁹https://github.com/yang-song/score_sde_pytorch

并实现了使用精确动作似然 $\pi_{\theta}(a_t|s_t)$ (3.1). 的策略梯度（单层 MDP）。

图 A10 展示了 DPPO 与使用精确似然估计的扩散策略梯度之间的比较。精确策略梯度在 Hopper-v2 中改进了基础策略，但未能超越 DPPO。由于需要反向传播通过常微分方程，它还需要更多的运行时间和 GPU 内存。在更具挑战性的 Can 任务中，其成功率降至零。此外，使用精确似然的策略梯度不具备 DPPO 所提供的灵活性，即微调少于 K 个去噪步骤或对早期去噪步骤进行折扣，而附录 C.2 已表明这些灵活性通常能提高微调效率。

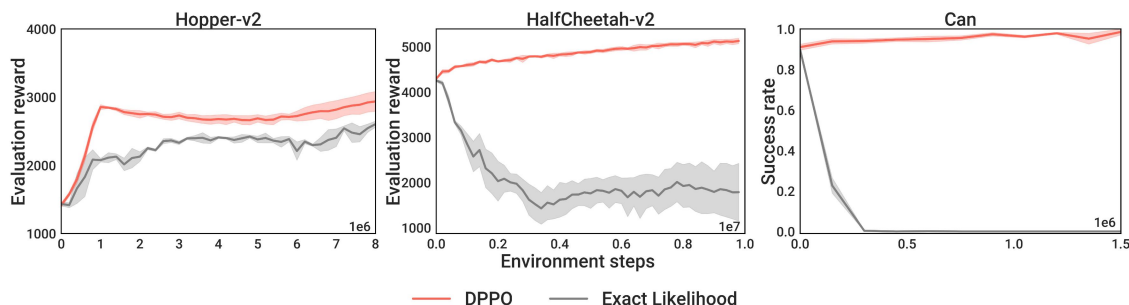


图 A10：与使用精确动作似然的扩散策略梯度进行比较。结果为 Hopper-v2 和 HalfCheetah-v2 上五个随机种子的平均值，以及 Can 上三个随机种子的平均值。

C.7 消融 DPPO 中的结构化探索

本文进一步论证了 DPPO（第 6 节）的结构化探索如何辅助强化学习微调。图 A8 将 DPPO 与高斯策略和 GMM 策略进行对比，显示 DPPO 轨迹实现了广泛覆盖并保持在专家数据流形附近；而在图 A11 中，我们对 DPPO 内部的结构化探索进行了消融。我们使用 DDIM [87] 使动作可以被确定性采样——这允许我们在不向中间去噪步骤添加任何噪声、仅向最终去噪动作（ $k = 0$ ）添加噪声的情况下采样轨迹，并将其与向所有去噪步骤添加噪声的 DPPO 进行对比。在两种情况下，我们考虑最小噪声水平 $\sigma_{\min}^{\text{exp}}$ 为 0.05 和 0.1。从图 A11 可以看出，在较高噪声水平下，DPPO 轨迹能很好地覆盖专家数据模式，而不会激进地探索新模式；而在仅向最后一步添加噪声的情况下，轨迹变得不够结构化，尤其是在 M3 中。随后，我们在 ROBOMIMIC 的 Can 和 Square 任务中运行两种探索方案，图 A3 右侧显示原始 DPPO 设置比仅在最后一步添加噪声时收敛更快。这一结果，加上第 5.3 节中显示 DPPO 比高斯策略和 GMM 策略具有更好样本效率的结果，展示了微调中结构化探索的优势。

D 挂钟时间报告

与其他基于扩散的强化学习算法比较 第 5.1 节。表 A1 和表 A2 显示了每个 OpenAI GYM 任务和 ROBOMIMIC 任务中使用的挂钟时间。在 GYM 任务中，DPPO 平均比 DAWR、DIPO 和 DQL 分别快 41%、37% 和 12% 完成训练，而这些方法都需要大量梯度更新才能稳定训练。QSM、DRWR 和 IDQL 分别比 DPPO 快 43%、33% 和 7%。ROBOMIMIC 任务的模拟成本更高，尤其是 Transport 任务，因此不同方法之间的挂钟时间差异较小。所有方法使用的时间相当，除了 DIPO 平均使用的时间略多。

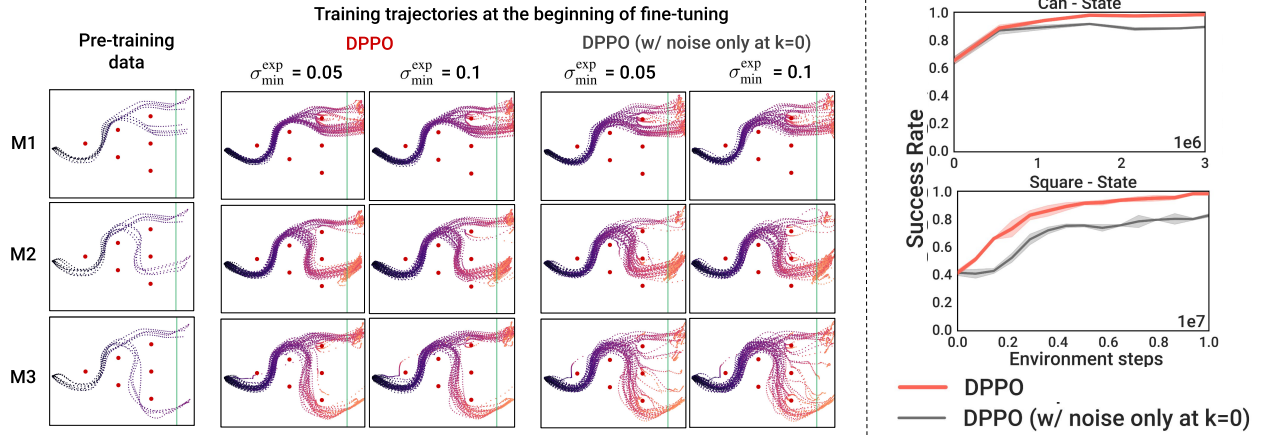


图 A11: DPPO 中结构化探索的消融。（左）在 Avoid 任务中，向所有去噪步骤添加噪声与仅向最后一步 $k = 0$ 添加噪声的采样轨迹。（右）结果在 Can 和 Square 任务中基于三个随机种子取平均。

方法	任务		
	Hopper-v2	Walker2D-v2	HalfCheetah-v2
DRWR	11.3	12.7	10.4
DAWR	30.4	30.7	27.1
DIPO	27.8	27.9	26.0
IDQL	16.3	16.1	15.5
DQL	20.5	20.5	17.6
QSM	9.6	9.9	9.7
DPPO	16.6	18.3	16.8

Table A1: **Wall-clock time** in seconds for a single training iteration in **OpenAI GYM tasks** when comparing diffusion-based RL algorithms. Each iteration involves 500 environment timesteps in each of the 40 parallelized environments running on 40 CPU threads and a NVIDIA RTX 2080 GPU (20000 steps total).

Method	Task			
	Lift	Can	Square	Transport
DRWR	32.5	39.5	59.8	346.1
DAWR	38.6	46.0	70.5	354.3
DIPO	43.9	51.6	73.3	359.7
IDQL	33.8	41.7	63.7	349.9
DQL	36.9	44.4	68.5	353.5
QSM	31.8	44.5	68.7	322.5
DPPO	35.2	42.0	65.6	350.3

表 A2: 在采用状态输入的 **ROBOMIMIC** 任务中，比较基于扩散的强化学习算法时，单次训练迭代的墙钟时间（秒）。每次迭代包含来自 50 个并行环境的各 4 个回合（Lift 和 Can 为 1200 个环境时间步，**Square** 为 1600 个，**Transport** 为 3200 个），这些环境运行在 50 个 CPU 线程和一块 NVIDIA L40 GPU 上（60000、80000、160000 步）。

与其他策略参数化和架构的比较 第 5.3 节和第 5.4 节。表 A3 和表 A4 分别展示了在采用状态或像素输入的几个 ROBOMIMIC 任务中微调所用的墙钟时间。高斯和 GMM 使用的时间相近，而变换器（Transformer）比多层感知机（MLP）略贵。在状态输入下，DPPO-MLP 平均比基线慢 24%、21%、24% 和 22%，原因是扩散采样更昂贵。DPPO-UNet 由于大量使用卷积层和归一化层需要更多时间，平均比 DPPO-MLP 慢 49%。在像素输入下，DPPO-ViT-MLP 平均比 Gaussian-ViT-MLP 慢 14%——差异比状态输入情况小，因为仿真中的渲染可能很昂贵。表 A5 展示了 FURNITURE-BENCH 任务中使用的墙钟时间。DPPO-UNet 平均比 Gaussian-MLP 慢 20%。

方法	任务			
	Lift	Can	Square	Transport
Gaussian-MLP	27.7	35.7	56.2	255.6
Gaussian-Transformer	29.8	37.1	57.8	266.1
GMM-MLP	28.0	36.2	55.2	254.5
GMM-Transformer	29.5	37.4	58.1	260.2
DPPO-MLP	35.6	43.3	65.0	350.5
DPPO-UNet	83.6	92.7	130.4	431.1

Table A3: **Wall-clock time** in seconds for a single training iteration in **ROBOMIMIC tasks with state input** when comparing policy parameterizations. Each iteration involves 4 episodes (1200 environment timesteps for Lift and Can, 1600 for Square, and 3200 for Transport) from each of the 50 parallelized environments running on 50 CPU threads and a NVIDIA L40 GPU (60000, 80000, 160000 steps).

Method	Task			
	Lift	Can	Square	Transport
Gaussian-ViT-MLP	153.6	173.1	277.0	770.0
DPPO-ViT-MLP	194.9	202.5	328.5	871.3

Table A4: **Wall-clock time** in seconds for a single training iteration in **ROBOMIMIC tasks with pixel input** when comparing policy parameterizations. Each iteration involves 4 episodes (1200 environment timesteps for Lift and Can, 1600 for Square, and 3200 for Transport) from each of the 50 parallelized environments running on 50 CPU threads and a NVIDIA L40 GPU (60000, 80000, 160000 steps).

Method	Task		
	One-leg	Lamp	Round-table
Gaussian-MLP	101.8	202.8	168.7
DPPO-UNet	148.4	258.2	188.6

表 A5: 在 **FURNITURE-BENCH** 任务中比较策略参数化时，单次训练迭代的墙钟时间（秒）。每次迭代包含来自 1000 个并行环境的各 1 个回合（One-leg 为 700 个环境时间步，Lamp 和 Round-table 为 1000 个），这些环境运行在一块 NVIDIA L40 GPU 上（700000、1000000、1000000 步）。

E 附加实验细节

Task / Dataset		Obs dim - State	Obs dim - Pixel	Act dim	T	Sparse reward ?
GYM	Hopper-v2	11	-	3	1000	No
	Walker2D-v2	17	-	6	1000	No
	HalfCheetah-v2	17	-	6	1000	No
FRANKA-KITCHEN	Kitchen-Complete-v0	60	-	9	280	Yes
	Kitchen-Partial-v0	60	-	9	280	Yes
	Kitchen-Mixed-v0	60	-	9	280	Yes
ROBOMIMIC, state input	Lift	19	-	7	300	Yes
	Can	23	-	7	300	Yes
	Square	23	-	7	400	Yes
	Transport	59	-	14	800	Yes
ROBOMIMIC, pixel input	Lift	9	96×96	7	300	Yes
	Can	9	96×96	7	300	Yes
	Square	9	96×96	7	400	Yes
	Transport	18	2×96×96	14	800	Yes
FURNITURE-BENCH	One-leg	58	-	10	700	Yes
	Lamp	44	-	10	1000	Yes
	Round-table	44	-	10	1000	Yes
D3IL	M1	4	-	2	100	Yes
	M2	4	-	2	100	Yes
	M3	4	-	2	100	Yes

表 A6: 所考虑的不同任务的比较。“观测维度-状态”: 状态观测输入的维度。“观测维度-状态”: 像素观测输入的维度。“动作维度-状态”: 动作空间的维度。 T : 一个回合中的最大步数。“稀疏奖励? ”: 训练中是否使用稀疏奖励而非密集奖励。

E.1 所有实验中使用的策略架构细节

多层感知机。在大多数实验中，我们采用带两层残差连接的多层感知机（MLP）作为策略头。对于基于扩散的策略，我们还使用一个小型多层感知机编码器处理状态输入，并使用另一个带正弦位置编码的小型多层感知机处理去噪时间步输入。它们的输出特征在送入多层感知机头之前进行拼接。由 Chi 等人提出的扩散策略 [20] 并未使用多层感知机作为扩散架构，但我们发现与 UNet 相比，它能提供相当（甚至更好）的预训练性能。

变换器。为了与第 5.3 节中的其他策略参数化方法进行比较，我们还考虑将变换器作为高斯和 GMM 基线的策略架构。我们仅考虑解码器。不使用随机失活。为动作块学习的位置嵌入作为序列输入解码器。

UNet。为了与第 5.3 节中的其他策略参数化方法进行比较，我们还考虑将 UNet [81] 作为扩散策略的一种可能架构。我们遵循 Chi 等人 [20] 的实现，对去噪时间步输入使用正弦位置编码，但在每个卷积块中对观测输入使用更大的多层感知机编码器。我们发现这一修改在更具挑战性的任务中很有帮助。

视觉变换器。对于第 5.3 节中基于像素的实验，我们在多层感知机头之前使用 Hu 等人 [40] 提出的基于视觉变换器 (ViT) 的图像编码器。本体感觉输入被附加到图像块的每个通道上。我们还遵循 [40]，对视觉变换器输出使用学习到的空间嵌入，以大幅减少特征数量，然后将这些特征送入下游的多层感知机头。

E.2 第 5.1 节中 GYM 任务和训练的补充细节

预训练。观测和动作使用预训练数据集的最小/最大统计量归一化到 $[0, 1]$ 。对于所有三个任务，策略训练 3000 个训练轮次，批大小为 128，学习率从 $1e-3$ 按余弦调度衰减到 $1e-4$ ，权重衰减为 $1e-6$ 。应用指数移动平均 (EMA)，衰减率为 0.995。

微调。第 5.1 节中的所有方法都使用相同的预训练策略。微调使用从 40 个并行 MuJoCo 环境 [92] 中采样的在线经验进行。奖励曲线显示在

图 5 通过运行微调后的策略进行评估，其中 $\sigma^{\text{exp}} = 0.001$ （即不添加额外噪声）进行 40 个回合。

每个回合在满足默认终止条件或达到 1000 个时间步时结束。详细超参数列于表 A7。

E.3 第 5.1 节中基于扩散的强化学习算法基线的描述

DRWR: 这是一种定制的奖励加权回归 (Reward-Weighted Regression, RWR) 算法 [70]，通过监督目标微调预训练的扩散策略 (Diffusion Policy, DP)，对导致更高奖励回报的动作赋予更高权重 r 。奖励按 β 缩放，指数化权重在 $w_{\max}$ 处截断。策略使用当前策略收集的经验进行更新（不使用前一次迭代的数据缓冲区），重放比率为 N_θ 。不学习评论家。

$$\mathcal{L}_\theta = \mathbb{E}^{\pi_{\theta, \varepsilon_t}} \left[\min(e^{\beta r_t}, w_{\max}) \|\varepsilon_t - \varepsilon_\theta(a_t^0, s_t, k)\|^2 \right].$$

DAWR: 这是一种定制的优势加权回归 (Advantage-Weighted Regression, AWR) 算法 [67]，在 DRWR 基础上构建，但使用时间差分引导的 [90] 优势估计替代方差较高的奖励回报，以提高训练稳定性和效率。

DAWR（和 DRWR）可视为在策略上带有 Kullback - Leibler (KL) 发散约束 [9, 67] 的情况下近似优化 (4.2)。优势按 β 缩放，指数化权重在 $w_{\max}$ 处截断。与 DRWR 不同，我们遵循 [67] 以离线方式训练行动者：近期经验保存在重放缓冲区 $\mathcal{D}$ 中，行动者以 N_θ 的重放比率进行更新。

$$\mathcal{L}_\theta = \mathbb{E}^{\mathcal{D}, \varepsilon_t} \left[\min(e^{\beta \hat{A}_\phi(s_t, a_t^0)}, w_{\max}) \|\varepsilon_t - \varepsilon_\theta(a_t^0, s_t, k)\|^2 \right].$$

评论家更新频率较低（我们发现扩散模型需要大量梯度更新来拟合动作），重放比率为 N_ϕ 。 $\mathcal{L}_\phi = \mathbb{E}^{\mathcal{D}} [\|\hat{A}_\phi(s_t, a_t^0) - A(s_t, a_t^0)\|^2]$ ，其中 A 使用 $\text{TD}(\lambda)$ 计算， λ 作为 λ_{DAWR} ，折扣因子为 γ_{ENV} 。

DIPO [103]: 该基线应用“动作梯度”，使用学习到的状态-动作 Q 函数更新重放缓冲区中保存的动作，然后在其上进行无加权的 DP 拟合。与 DAWR 类似，近期经验保存在重放缓冲区 $\mathcal{D}$ 中。缓冲区中的动作 ($k = 0$) 以学习率 α_{DIPO} 更新 M_{DIPO} 次迭代。

$$a_t^{m+1, k=0} = a_t^{m, k=0} + \alpha_{\text{DIPO}} \nabla_\phi \hat{Q}_\phi(s_t, a_t^{m, k=0}), m = 0, \dots, M_{\text{DIPO}} - 1.$$

然后以重放比率 γN_θ 更新演员。

$$\mathcal{L}_\theta = \mathbb{E}^{\mathcal{D}} [\|\varepsilon_t - \varepsilon_\theta(a_t^{M_{\text{DIPQ}}, k=0}, s_t, k)\|^2].$$

评论家被训练以最小化贝尔曼残差，重放比率为 γN_ϕ 。同时使用双 Q 学习 applied.

$$\mathcal{L}_\phi = \mathbb{E}^{\mathcal{D}} [\|(R_t + \gamma_{\text{ENV}} \hat{Q}_\phi(s_{t+1}, \bar{\pi}_\theta(a_{t+1}^{k=0}|s_{t+1})) - \hat{Q}_\phi(s_t, a_t^{m=0, k=0})\|^2]$$

IDQL [35]: 该基线学习状态-动作 Q 函数和状态 V 函数，以在 DP 采样的动作中进行选择。DP 在新样本上进行无加权拟合。同样，近期经验保存在重放缓冲区 $\mathcal{D}$ 中。状态值函数通过期望损失更新以匹配期望 Q 值，重放比率为 γN_ψ 。

$$\mathcal{L}_\psi = \mathbb{E}^{\mathcal{D}} [|\pi_{\text{IDQL}} - \mathbb{1}(\hat{Q}_\phi(s_t, a_t^0) < \hat{V}_\psi^2(s_t))|].$$

值函数用于以重放比率 γN_ϕ 更新 Q 函数。

$$\mathcal{L}_\phi = \mathbb{E}^{\mathcal{D}} [\|(R_t + \gamma_{\text{ENV}} \hat{V}_\psi(s_{t+1}) - \hat{Q}_\phi(s_t, a_t^0)\|^2].$$

演员对所有采样经验进行无加权拟合，重放比率为 γN_θ 。

$$\mathcal{L}_\theta = \mathbb{E}^{\mathcal{D}} [\|\varepsilon_t - \varepsilon_\theta(a_t^0, s_t, k)\|^2].$$

在推理时，从演员采样 M_{IDQL} 个动作。训练时，基于采样动作的 Q 值与当前状态的 V 值之间的差异应用玻尔兹曼探索。评估时，选择 Q 下的贪心动作。

双 Q 学习 (Double Q-learning) [100]: 该基线学习状态-动作 Q 函数，并通过整个演员（包含多个去噪步骤）反向传播评论家的梯度，类似于通常的 Q 学习。同样，近期经验保存在回放缓冲区 $\mathcal{D}$ 中。然后使用监督损失和价值损失更新演员，回放比率为 γN_θ 。

$$\mathcal{L}_\theta = \mathbb{E}^{\mathcal{D}} [\|\varepsilon_t - \varepsilon_\theta(a_t^0, s_t, k)\|^2 - \alpha_{\text{DQL}} \hat{Q}_\phi(s_t, \bar{\pi}_\theta(a_t^0|s_t))],$$

其中 α_{DQL} 是加权系数。评论家被训练以最小化贝尔曼残差，回放比率为 γN_ϕ 。同时应用双 Q 学习。

$$\mathcal{L}_\phi = \mathbb{E}^{\mathcal{D}} [\|(R_t + \gamma_{\text{ENV}} \hat{Q}_\phi(s_{t+1}, \bar{\pi}_\theta(a_{t+1}^0|s_{t+1})) - \hat{Q}_\phi(s_t, a_t^0)\|^2]$$

QSM [74]: 该基线学习状态-动作 Q 函数，然后将扩散演员的分数与 Q 函数的梯度对齐来更新演员。同样，近期经验保存在回放缓冲区 $\mathcal{D}$ 中。评论家被训练以最小化贝尔曼残差，回放比率为 γN_ϕ 。同时应用双 Q 学习。

$$\mathcal{L}_\phi = \mathbb{E}^{\mathcal{D}} [\|(R_t + \gamma_{\text{ENV}} \hat{Q}_\phi(s_{t+1}, \bar{\pi}_\theta(a_{t+1}^0|s_{t+1})) - \hat{Q}_\phi(s_t, a_t^0)\|^2].$$

演员按如下方式更新，回放比率为 γN_θ 。

$$\mathcal{L}_\theta = \mathbb{E}^{\mathcal{D}} [\|\alpha_{\text{QSM}} \nabla_a \hat{Q}_\phi(s_t, a_t) - (-\varepsilon_\theta(a_t^0, s_t, k))\|^2],$$

其中 α_{QSM} 缩放梯度。 ε_θ 前的负号来自在去噪过程中对均值 μ 取梯度。

E.4 第 5.2 节中强化学习微调算法基线的描述 在本小节中，我们详细介绍基线 RLPD、Cal-QL 和 IBRL。所有策略 π_θ 均参数化为单峰高斯分布。

RLPD [7]: 该基线基于软演员评论家 (Soft Actor Critic, SAC, Haarnoja 等人 [33]) ——它学习熵正则化的状态-动作 Q 函数，然后通过关于动作最大化 Q 函数来更新演员。回放缓冲区 $\mathcal{D}$ 用离线数据初始化，在线样本被添加到 $\mathcal{D}$ 中。每次梯度更新使用 50/50 混合的离线和在线数据批次。使用 N_{critic} 个评论家的集成，在每个梯度步骤随机选择两个评论家。评论家被训练以最小化贝尔曼残差，回放比率为 N_ϕ ：

$$\mathcal{L}_\phi = \mathbb{E}^{\mathcal{D}} [\|(R_t + \gamma_{\text{ENV}} \hat{Q}_{\phi'}(s_{t+1}, \pi_\theta(a_{t+1}|s_{t+1})) - \hat{Q}_\phi(s_t, a_t)\|^2].$$

目标评论家参数 ϕ' 延迟更新。演员最小化以下损失，回放比率为 1：

$$\mathcal{L}_\theta = \mathbb{E}^{\mathcal{D}} [-\hat{Q}_\phi(s_t, a_t) + \alpha_{\text{ent}} \log \pi_\theta(a_t|s_t)],$$

其中 α_{ent} 为熵系数（如 SAC 中自动调节，初始值为 1）。

校准 Q 学习 (Cal-QL) [62]: 该基线先在离线阶段训练策略 μ 与动作价值函数 Q^μ ，随后进入在线阶段。离线阶段仅采样离线数据进行梯度更新，在线阶段则按 50/50 混合采样离线与在线数据。评论家通过最小化以下损失（贝尔曼残差与校准 Q 学习）进行训练：

$$\begin{aligned} \mathcal{L}_\phi = & \mathbb{E}^{\mathcal{D}} [\|(R_t + \gamma_{\text{ENV}} \hat{Q}_{\phi'}(s_{t+1}, \pi_\theta(a_{t+1}|s_{t+1}))) - \hat{Q}_\phi(s_t, a_t)\|^2] \\ & + \beta_{\text{cql}} (\mathbb{E}^{\mathcal{D}} [\max(Q_\phi(s_t, a_t), V(s_t))] - \mathbb{E}^{\mathcal{D}} [Q_\phi(s_t, a_t)]), \end{aligned}$$

其中 β_{cql} 为贝尔曼残差与校准 Q 学习之间的权重系数， $V(s_t)$ 使用蒙特卡洛回报估计。目标评论家参数 ϕ' 延迟更新。演员最小化以下损失： $\mathcal{L}_\theta = \mathbb{E}^{\mathcal{D}} [-\hat{Q}_\phi(s_t, a_t) + \alpha_{\text{ent}} \log \pi_\theta(a_t|s_t)]$ ，其中 α_{ent} 为熵系数（如 SAC 中自动调节，初始值为 1）。

IBRL [40]: 该基线首先使用行为克隆预训练策略 μ_ψ ，微调时训练强化学习策略 π_θ ，其初始化为 μ_ψ 。微调过程中，近期经验存入回放缓冲区 $\mathcal{D}$ 。使用 N_{critic} 个评论家的集成，每个梯度步骤随机选择两个评论家。评论家以回放比率 N_ϕ 最小化贝尔曼残差进行训练：

$$\mathcal{L}_\phi = \mathbb{E}^{\mathcal{D}} [\|(R_t + \gamma_{\text{ENV}} \max_{a' \in \{a^{IL}, a^{RL}\}} \hat{Q}_{\phi'}(s_{t+1}, a') - \hat{Q}_\phi(s_t, a_t)\|^2]$$

其中 $a^{IL} = \mu_\psi(s_{t+1})$ （无噪声）与 $a^{RL} \sim \pi_{\theta'}(s_{t+1})$ ， $\pi_{\theta'}$ 为目标演员。目标评论家参数 ϕ' 延迟更新。演员以回放比率 1 最小化以下损失：

$$\mathcal{L}_\theta = -\mathbb{E}^{\mathcal{D}} [\hat{Q}_\phi(s_t, a_t)].$$

目标演员参数 θ' 同样延迟更新。

E.5 第 5.2 节中 FRANKA-KITCHEN 任务与训练的补充细节

任务。本文考虑来自 D4RL 基准 [29] 的三种设置：（1）Kitchen-Complete-v0 包含完成整个任务（四个子任务）的演示，（2）Kitchen-Partial-v0 包含一些完整演示和许多仅完成子任务的演示，以及（3）Kitchen-Mixed-v0 仅包含不完整的演示。

预训练。观测和动作使用预训练数据集的最小/最大统计量归一化到 $[0, 1]$ 。不使用历史观测（本体感觉或真实物体状态）。所有策略均以批大小 128、学习率 $1e-4$ （按余弦调度衰减至 $1e-5$ ）和权重衰减 $1e-6$ 进行训练。DPPO 策略训练 8000 个训练轮次。对于 IBRL 和 **Cal-QL**，本文遵循原始实现中的超参数——IBRL 建议在预训练期间使用（1）更宽的多层感知机层和（2）随机失活，本文也遵循此建议。本文对 DPPO 使用 $T_a = 4$ ；本文还尝试对 IBRL、RLPD 和 **Cal-QL** 使用相同的动作块大小，但发现对于所有这些方法 $T_a = 1$ 能带来更好的性能。

微调。使用 DPPO 时，策略利用从 40 个并行 MuJoCo 环境 [92] 采样的在线经验进行微调，而基线仅使用一个环境（与其原始实现一致）。回合在达到最大回合长度（见表 A6）或所有四个子任务完成时终止。详细超参数列于表 A8——本文遵循基线原始实现中的超参数选择。

更大的方差 图 6 中的 DPPO。图 6 显示，与 **Cal-QL** 相比，DPPO 在 Kitchen-Partial-v0 任务上的归一化得分表现出更大的方差。这是因为 DPPO 在单个随机种子中要么解决 3/4 个子任务，要么解决 2/4 个子任务（单个种子内评估回合的方差较低），但在不同种子之间方差较高；而 **Cal-QL** 在单个种子内的评估回合中方差较高，但在种子间平均方差较低。这也凸显了 DPPO 的一个显著特性：Kitchen-Partial-v0 和 Kitchen-Mixed-v0 的轨迹仅完成部分子任务，因此具有高度多模态性。扩散策略有时难以从预训练中学习所有模式，并且由于 DPPO 直接微调预训练策略，它可能在微调时无法收敛到 100% 的成功率。相比之下，**Cal-QL** 在微调期间以离策略方式从所有离线数据中学习，因此对预训练性能不太敏感。尽管如此，在离线数据能够一致完成任务但质量参差不齐的情况下（ROBOMIMIC 和 Kitchen-Complete-v0，我们认为这在当前机器人操作范式中更为现实），DPPO 在图 6 中展现出比 **Cal-QL** 和其他基线更好的最终性能。

E.6 第 5.3 节中 ROBOMIMIC 任务和训练的补充细节

任务。我们考虑 ROBOMIMIC 基准 [60] 中的四个任务：（1）举起：从桌上举起一个立方体，（2）罐子：拾起一个可乐罐并将其放入目标箱中，（3）方形：拾起一个方形螺母并将其套在杆上，（4）搬运：两个机械臂移除箱盖，拾起并放置一个立方体，然后将一把锤子从一个容器转移到另一个容器。

预训练。ROBOMIMIC 为每个任务提供了带有噪声人类演示的多人类（MH）数据集，我们使用该数据集来预训练策略。观测和动作使用预训练数据集的最小/最大统计量归一化到 $[0, 1]$ 区间。不使用历史观测（像素、本体感知或真实物体状态）。所有策略均以批大小 128、学习率 $1e-4$ （按余弦调度衰减至 $1e-5$ ）和权重衰减 $1e-6$ 进行训练。基于扩散的策略训练 8000 个训练轮次，

而高斯策略和高斯混合模型策略训练 5000 个训练轮次——我们发现扩散模型需要更多的梯度更新才能很好地拟合数据。

微调。随后，基于扩散、高斯和混合高斯模型（GMM）的预训练策略利用从 50 个并行化 MuJoCo 环境中采样的在线经验进行微调 [92]。图 5、图 7 和图 A9 中展示的成功率曲线是通过运行微调后的策略（使用 σ^{exp} ，即不添加额外噪声）进行 $\epsilon = 0.001$ 评估的共进行 50 个回合。回合仅在达到最大回合长度时终止（见表 A6）。详细的超参数列于表 A9。

像素训练。在 Lift 和 Can 任务中使用腕部相机视角，在 Square 任务中使用第三人称相机视角，在 Transport 任务中使用两个机器人肩部相机视角。在预训练和微调期间，对相机图像应用随机平移数据增强。微调中使用梯度累积，以便在 GPU 上容纳与状态输入训练相同的批大小。详细的超参数列于表 A10。

E.7 第 5.3 节中策略参数化基线的描述

高斯。我们考虑具有对角协方差的单峰高斯分布，这是强化学习中最常用的策略参数化方式。每个动作维度的标准差 σ_{Gau} 在预训练期间固定；我们也尝试从数据集中学习 σ_{Gau} ，但发现训练非常不稳定。在微调期间， σ_{Gau} 从相同的固定值开始学习，并被裁剪在 0.01 和 0.2 之间。此外，我们将采样动作裁剪在均值 3 个标准差范围内。如附录 B 所述，我们根据每个任务中的经验裁剪比例选择 PPO 裁剪比率 ϵ 。此设置也用于 FURNITURE-BENCH 实验。我们注意到，我们在调整高斯基线上花费了大量精力，其结果在长时程操作任务的强化学习训练中属于已知最佳结果之一（超出了我们最初的预期），例如，在低随机性的 Lamp 任务中达到 $\sim 100\%$ 的成功率。

混合高斯模型（GMM）。我们还考虑将混合高斯模型作为策略参数化方式。我们用 M_{GMM} 表示混合成分的数量。每个混合成分中每个动作维度的标准差 σ_{GMM} 在预训练期间也固定。同样，在微调期间， σ_{GMM} 从相同的固定值开始学习，并被裁剪在 0.01 和 0.2 之间。

E.8 第 5.4 节中 FURNITURE-BENCH 任务与训练的补充细节

任务。我们考虑 FURNITURE-BENCH 基准 [36] 中的三个任务：（1）单腿：将桌面放置在夹具角落，抓取并插入桌腿，然后拧紧桌腿，完成桌子单腿的组装；（2）台灯：将灯座放置在夹具角落，抓取、插入并拧紧灯泡，最后放置灯罩；（3）圆桌：将圆形桌面放置在夹具角落，插入并拧紧桌腿，然后插入并拧紧桌底座。仿真中的可视化轨迹见图 A12。

预训练。预训练数据集是在模拟环境中使用 SpaceMouse¹⁰，a 六自由度输入设备采集的。模拟器以 10Hz 运行。在每个时间步，我们读取 SpaceMouse 的状态为 $\delta \mathbf{a} = [\Delta x, \Delta y, \Delta z, \Delta \text{横滚}, \Delta \text{俯仰}, \Delta \text{yaw}]$ ，在传入环境步骤之前将其转换为四元数，并与当前观测一起作为动作存储在轨迹中。如果 $|\Delta \mathbf{a}_i| < \epsilon \forall i$ ¹⁰
<https://3dconnexion.com/us/product/spacemouse-wireless/>

对于某个定义无操作阈值的小量 $\varepsilon = 0.05$ ，我们不记录任何动作，也不将其传入环境。丢弃无操作对于让策略从演示中有效学习至关重要。当收集到所需数量的演示（通常为 50 个）后，我们处理动作，将 SpaceMouse 存储的增量动作转换为绝对位姿动作，方法是在每个时间步将增量动作应用于当前末端执行器位姿。观测和动作使用预训练数据集的最小/最大统计量归一化到 $[-1, 1]$ 。不使用历史观测（本体感知或真实物体状态），即仅将当前观测传递给策略。所有策略均以批大小 256、学习率 $1e-4$ （按余弦调度衰减至 $1e-5$ ）和权重衰减 $1e-6$ 进行训练。基于扩散的策略训练 8000 个训练轮次，而高斯策略训练 3000 个训练轮次。高斯策略容易过拟合预训练数据集，而基于扩散的策略更具鲁棒性。高斯策略还需要非常大的多层感知机（ ~ 1000 万参数）才能很好地拟合数据。

微调。随后，利用从 1000 个并行化的艾萨克健身房（IsaacGym）环境中采样的在线经验，对基于扩散和高斯预训练的策略进行微调 [59]。图 8 所示的成功率曲线是通过运行微调后的策略 $\sigma_{\min}^{\text{exp}} = 0.001$ （即不添加额外噪声）进行 1000 个回合评估得到的。回合仅在达到最大回合长度时终止（见表 A6）。详细超参数列于表 A11。我们发现较小的探索噪声（将 $\sigma_{\min}^{\text{exp}}$ 和 σ_{Gau} 设为 0.04）对于预训练策略在微调初期获得非零成功率是必要的。

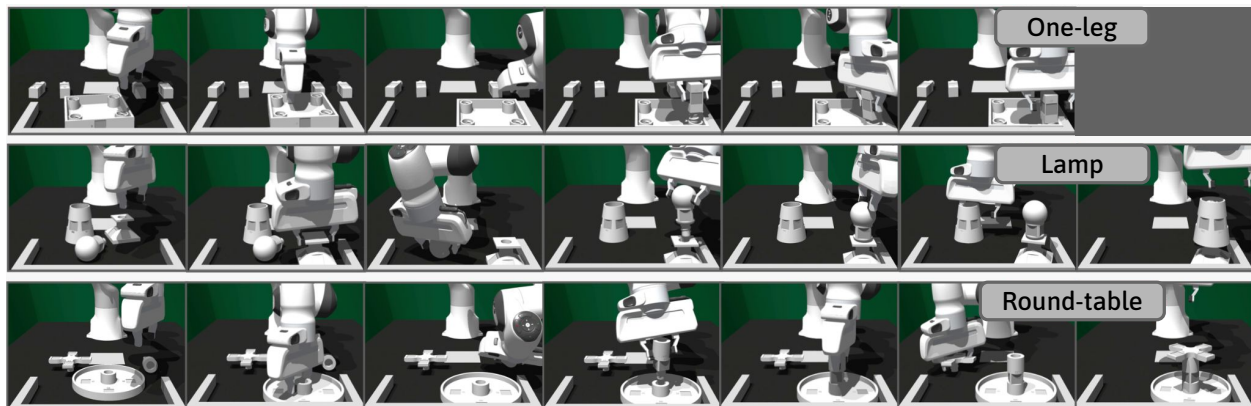


图 A12: 模拟家具基准（FURNITURE-BENCH）任务中的代表性展开轨迹。

硬件配置——机器人控制。

所使用的实体机器人为 Franka Emika Panda 机械臂。策略输出机器人基座坐标系下的一系列期望末端执行器位姿，以控制机器人。这些位姿通过微分逆运动学转换为关节位置目标。我们计算期望末端执行器速度，即期望位姿与当前位姿之差除以时间步长 $dt = 1/10$ 。随后利用雅可比矩阵将其转换为期望关节速度，并对当前关节位置与期望速度进行一阶积分，计算期望关节位置。所得关节位置目标传递给由 Polymetis [55] 提供的底层关节阻抗控制器，以 1kHz 频率运行。

硬件配置——状态估计。

为了在真实硬件上部署基于状态（state-based）的策略，我们利用四月标签（AprilTags）[98] 进行零件姿态估计。家具基准（FURNITURE-BENCH）[36] 任务套件为每个零件提供四月标签，并提供从标签检测估计零件姿态的代码。该过程包括几个步骤：（1）在相机帧中检测标签，（2）将标签检测映射到机器人帧以实现策略兼容，（3）利用模拟器中标签与物体中心之间的已知偏移，以及（4）使用一个

位于机器人基座已知相对位置的四月标签（AprilTag）。尽管总体准确率尚可，但检测结果可能存在噪声，尤其是在移动或部分遮挡时，单腿任务正是如此。由于该任务要求高精确率，我们发现以下方法有助于使估计足够可靠：

- **摄像头覆盖范围：**我们发现检测质量对相机与标签之间的距离和角度敏感。这一问题可能源于 RealSense D435 相机图像质量与清晰度一般，以及标签尺寸相对较小。为解决此问题，我们选择在场景周围大致均匀地布置 4 台相机，以确保至少有一台相机能够清晰观察到所有部件上的标签（即尽可能接近且正对标签）。为确定最佳相机位置，我们首先在场景的四个基本方向各放置一台相机。随后，调整每台相机的姿态，使其尽可能靠近物体，同时仍覆盖必要的工作空间并捕捉用于校准的基准标签。在场景中移动机械臂以避免最严重的遮挡也有帮助。

- **光照：**即使改善了相机覆盖范围和布置，检测质量仍依赖于清晰的图像。我们发现适当的光照有助于提高图像质量。特别是，场景周围应光照充足且均匀，同时避免在标签或桌面上产生反射。

- **滤波：**不良检测有时会导致位姿估计结果显著偏离真实位姿，即从一帧到下一帧跳跃数厘米。这种情况通常仅发生在孤立帧上，因此在“接受”给定检测之前，我们检查新位置和方向是否在先前接受位姿的 5 厘米和 20 度范围内。此外，我们使用简单的指数平均（其中 $\alpha = 0.25$ ）对检测结果进行低通滤波，以平滑高频噪声。

- **平均：**物体具有多个标签，可被多台相机检测到。执行滤波步骤后，我们对同一物体在不同标签和相机下的所有位姿估计进行平均，这也有助于平滑噪声。然而，仅靠平均并不能完全消除单次检测出现大幅跳跃的情况，因为这会严重扭曲平均值，因此仍需要滤波步骤。拥有多台相机也有利于此步骤，因为它提供了更多可平均的检测结果。

- **缓存手中部件位姿：**任务中实现良好检测的一个特别困难阶段是机器人将桌腿从初始位置搬运到桌面进行插入时。主要问题在于移动可能导致图像模糊，抓取可能造成遮挡。因此，我们发现假设零件一旦被机器人抓住，在夹爪打开前不会在抓取中移动是有帮助的。这样，我们可以在物体被完全抓住后“缓存”零件相对于末端执行器的位姿，并在移动过程中使用该位姿，而不是依赖检测。

- **归一化陷阱与裁剪：**我们通常对状态观测使用最小-最大归一化，以确保观测值在 $[-1, 1]$ 范围内。桌面零件在 z 方向演示数据中移动很小，这意味着得到的归一化极限（数据的最小值和最大值）可能非常接近， $x_{\max} - x_{\min} \approx 0$ 。在这些紧密的极限下，真实世界检测中的噪声会被大幅放大，如所示。因此，确

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \text{ 保归一化范围合理。作为额外保障，将数据裁剪到 } [-1, 1] \text{ 也有帮助。}$$

- **仅估计必要状态：**尽管单腿任务有 5 个零件，但只有 2 个被操作。仅估计这些零件的位姿可以消除大量噪声。特别是，未使用的 3 条腿和障碍物（U 形夹具）的位姿可以设为数据集中的任意值。

- **可视化调试：**我们广泛使用可视化工具 MeshCat ¹¹ 进行状态估计调试。该工具可以轻松可视化场景中所有相关物体的位姿，¹¹ <https://github.com/meshcat-dev/meshcat>

如机器人末端执行器和零件，这使得检查实现远比查看原始数字容易。

硬件评估。我们对每种方法进行 20 次试验。采用单盲模型选择流程：每次试验开始时，先随机化初始状态。然后随机选择一种方法并执行，但实验者不知道使用的是哪个模型。记录每次试验的成功与失败，待所有试验完成后，再汇总各模型的统计数据。

用于仿真到现实迁移的领域随机化。为促进仿真到现实迁移，我们在仿真训练中额外应用领域随机化。记录硬件在无任何机器人运动时的观测噪声范围，然后在仿真中对状态观测施加相同幅度的噪声。我们发现硬件中的状态估计对物体高度特别敏感。此外，对从 DPPO 采样的动作施加随机噪声（零均值，标准差 0.03），以模拟不完美的底层控制器；我们发现对高斯策略添加此类噪声会导致任务成功率为零，而 DPPO 对此具有鲁棒性（另见第 6 节讨论）。

用于高斯基线的行为克隆正则化损失。由于微调后的高斯策略表现出非常抖动的行为，并在真实评估中导致零成功率，我们进一步实验在微调高斯基线时添加行为克隆（Behavior Cloning）正则化损失。组合损失如下

$$\mathcal{L}_{\theta, +BC} = \mathcal{L}_{\theta} - \alpha_{BC} \mathbb{E}^{\pi_{\theta_{old}}} \left[\sum_{k=0}^{K-1} \log \pi_{\theta_{pre-trained}}(a_t^k | a_t^{k+1}, s_t) \right],$$

其中 $\pi_{\theta_{pre-trained}}$ 是冻结的仅行为克隆策略。额外项鼓励微调策略新采样的动作在仅行为克隆策略下保持高似然。我们设置 $\alpha_{BC} = 0.1$ 。然而，尽管该正则化缩小了仿真到现实差距，它也显著限制了微调，导致微调策略饱和在 53% 的成功率，如图 8 所示。

E.9 第 6 节中 D3IL 的避障任务及训练的补充细节

预训练。我们根据三种设置 M1、M2 和 M3 划分 D3IL 的原始数据集；在每种设置中，观测和动作使用最小/最大统计量归一化到 $[0, 1]$ 。所有策略均以批大小 16（由于数据集较小）、学习率 $1e-4$ 按余弦调度衰减至 $1e-5$ 、权重衰减 $1e-6$ 进行训练。基于扩散的策略训练约 15000 个训练轮次，而高斯和 GMM 策略训练约 10000 个训练轮次；我们手动检查不同预训练检查点的轨迹，并选择在视觉上最匹配专家数据的检查点。

微调。基于扩散、高斯和混合高斯模型（GMM）的预训练策略随后利用从 50 个并行化的 MuJoCo 环境中采样的在线经验进行微调 [92]。图 11 和图 A8 中展示的奖励曲线是通过运行微调后的策略进行评估得到的，评估时使用与训练中相同的探索噪声，共进行 50 个回合；我们选择使用训练（而非评估）设置，因为高斯策略仅在存在训练噪声时才表现出多模态性。回合仅在达到 100 步时终止。

在微调过程中添加了动作噪声。在图 11 左侧，我们证明了当在微调期间向采样动作添加噪声时，DPPO 表现出更强的训练稳定性。噪声从第 5 次迭代开始添加。它从均匀分布中采样，其下限在 5 次迭代内线性上升至 0.1，上限线性上升至 0.2。从第 10 次迭代到微调结束，这些界限保持不变。

E.10 列出的训练超参数

Method	Parameter	Task(s)			
		GYM	Lift, Can	Square	Transport
Common	γ_{ENV}^{exp}	0.99	0.999	0.999	0.999
	σ_{min}^{prob}	0.1	0.1	0.1	0.08
	σ_{min}			0.1	
	T_p	4	4	4	8
	T_a	4	4	4	8
	K			20	
	Actor learning rate	1e-4 for DPPO and 1e-5 for others (tuned from 1e-4 to 1e-5)			
	Critic learning rate (if applies)	1e-3			
	Actor MLP dims	[512, 512, 512]	[512, 512, 512]	[1024, 1024, 1024]	[1024, 1024, 1024]
	Critic MLP dims (if applies)	[256, 256, 256]			
DRWR	β			10	
	w_{max}			100	
	N_θ			16	
	Batch size			1000	
DAWR	β			10	
	w_{max}			100	
	λ_{DAWR}			0.95	
	N_θ			64	
	N_ϕ			16	
	Buffer size	200000	120000	120000	120000
DIPO	Batch size			1000	
	α_{DIPO}			1e-4	
	M_{DIPO}			10	
	N_θ			64	
	Buffer size			1000000	
	Batch size			1000	
IDQL	M_{IDQL}	20	10	10	10
	N_θ			128	
	N_ϕ			128	
	Buffer size	1000000	250000	250000	250000
	Batch size			1000	
DQL	α_{DQL}			1	
	N_θ			16	
	N_ϕ			16	
	Buffer eize			1000000	
	Batch size			1000	
QSM	α_{QSM}			10	
	N_θ			16	
	N_ϕ			16	
	Buffer size	1000000	250000	250000	250000
	Batch size			1000	
DPPO	$\gamma_{DENOISE}$			0.99	
	GAE λ			0.95	
	N_θ	5	10	10	10
	N_ϕ	5	10	10	10
	ε			0.01	
	Batch size	50000	7500	10000	10000
	K'			10	

表 A7: 在比较基于扩散的强化学习方法时，用于 OpenAI GYM 和 ROBOMIMIC 任务的微调超参数。我们首先列出共享的超参数，然后列出各方法特定的超参数。

		Task(s)				
方法	参数	半猎豹-v2	厨房-完整-v0	厨房-部分-v0	厨房-混合-v0	
Common	γ_{ENV}					0.99
RLPD	T_p					1
	T_a					1
	N_ϕ	2 3		10310		10
	N_{critic}	10		5		5
	Batch size					256
Cal-QL	T_p					1
	T_a					1
	β_{cql}					5
	Batch size					256
IBRL	T_p					1
	T_a					1
	N_ϕ					5
	N_{critic}					5
	Batch size					256
DPPO	T_p	1	4		4	4
	T_a	1	4		4	4
	σ_{min}^{exp}					0.1
	σ_{min}^{min}					0.1
	$\gamma_{DENOISE}$					0.99
	GAE λ					0.95
	N_θ	5	10		10	10
	N_ϕ	5	10		10	10
	ε					0.01
	Batch size	10000	5600		5600	5600
	K					20
	K'					10

Method	Parameter	Can, PH	Square, PH	Can, MH	Square, MH
Common	γ_{ENV}				0.999
	T_a				1
	T_a				1
RLPD	N_ϕ				3
	N_{critic}				5
	Batch size				256
Cal-QL	β_{cql}				5
	Batch size				256
IBRL	N_ϕ				3
	N_{critic}				5
	Batch size				256
DPPO	σ_{min}^{exp}				0.1
	σ_{min}^{min}				0.1
	$\gamma_{DENOISE}$	0.9	0.9		0.99
	GAE λ				0.95
	N_θ				10
	N_ϕ				10
	ε				0.01
	Batch size	6000	15000	8000	20000
	K				20
	K'				10

表 A8: 在比较演示增强强化学习方法时, 用于 HalfCheetah-v2、FRANKA-KITCHEN、Can 和 Square (PH 或 MH 数据集) 的微调超参数。我们首先列出所有方法共享的超参数, 然后列出各方法特定的超参数。

Method	Parameter	任务		
		Lift, Can	Square	Transport
Common	γ^{ENV}		0.999	
	T_a	4	4	8
	Actor learning rate	1e-4	1e-5	1e-5 (decayed to 1e-6)
	Critic learning rate		1e-3	
	GAE λ		0.95	
	N_θ	10	10	8
	N_ϕ	10	10	8
	ε		0.01 (annealed in DPPO)	
Gaussian, Common	σ_{Gau}	0.1	0.1	0.08
	Batch size	7500	10000	10000
Gaussian-MLP	Model size	552K	2.15M	1.93M
Gaussian-Transformer	Model size	675K	1.86M	1.87M
GMM, Common	M_{GMM}		5	
	σ_{GMM}	0.1	0.1	0.08
	Batch size	7500	10000	10000
	Model size	1.15M	4.40M	4.90M
GMM-MLP	Model size			
GMM-Transformer	Model size	680K	1.87M	1.89M
DPPO , Common	γ^{DENOISE}		0.99	
	$\sigma_{\min}^{\text{exp}}$	0.1	0.1	0.08
	$\sigma_{\min}^{\text{prob}}$	0.1	0.1	0.1
	K		20	
	K'		10	
	Batch size	75000	100000	100000
DPPO -MLP	Model size	576K	2.31M	2.43M
DPPO -UNet	Model size	652K	1.62M	1.68M

表 A9: 在比较策略参数化时, 用于具有状态输入的 ROBOMIMIC 任务的微调超参数。我们首先列出所有方法共享的超参数, 然后列出各方法特定的超参数。由于不同的策略参数化使用不同的神经网络架构, 我们在此列出总模型大小, 而不是诸如多层感知机 (MLP) 维度之类的细节。

Method	Parameter	任务		
		Lift, Can	Square	Transport
Common	γ_{ENV}		0.999	
	T_a	4	4	8
	Actor learning rate	1e-4	1e-5	1e-5 (decayed to 1e-6)
	Critic learning rate		1e-3	
	GAE λ		0.95	
	N_θ	10	10	8
	N_ϕ	10	10	8
	ε		0.01 (annealed in DPPO)	
Gaussian-ViT-MLP	Model size	1.03M	1.03M	1.93M
	σ_{Gau}	0.1	0.1	0.08
	Batch size	7500	10000	10000
DPPO -ViT-MLP	Model size	1.06M	1.06M	2.05M
	$\gamma_{DENOISE}$		0.9	
	σ_{min}^{exp}	0.1	0.1	0.08
	σ_{min}^{prob}		0.10	
	K		100	
	K'		5 (DDIM)	
	Batch size	37500	50000	50000

表 A10: 在比较策略参数化时, 针对像素输入的机器人模仿任务微调超参数。首先列出所有方法共用的超参数, 然后列出各方法特有的超参数。由于不同策略参数化使用不同的神经网络架构, 此处列出总模型规模, 而非多层感知机维度等细节。

Method	Parameter	任务		
		One-leg	Lamp	Round-table
Common	γ_{ENV}		0.999	
	T_a		8	
	Actor learning rate		1e-5 (decayed to 1e-6)	
	Critic learning rate		1e-3	
	GAE λ		0.95	
	N_θ		5	
	N_ϕ		5	
	ε		0.001	
Gaussian-MLP	Model size	10.64M	10.62M	10.62M
	σ_{Gau}		0.04	
	Batch size		8800	
DPPO -UNet	Model size	6.86M	6.81M	6.81M
	$\gamma_{DENOISE}$		0.9	
	σ_{min}^{exp}		0.04	
	σ_{min}^{prob}		0.1	
	K		100	
	K'		5 (DDIM)	
	Batch size		44000	

表 A11: 在比较策略参数化时, 针对家具基准任务的微调超参数。首先列出所有方法共用的超参数, 然后列出各方法特有的超参数。